

Master's Thesis

Practical Regularized Quasi-Newton
Methods with Inexact Function Values
(不正確な関数値下における
実用的な正則化準ニュートン法)

48-246226 Hiroki Hamaguchi

Supervisor Professor Takeda Akiko

January, 2026

Department of Mathematical Informatics
Graduate School of Information Science and Technology
The University of Tokyo

Abstract

Many practical optimization problems involve objective function values that are corrupted by unavoidable numerical errors. In smooth unconstrained optimization, quasi-Newton methods combined with line search are widely used due to their efficiency and scalability. These methods implicitly assume accurate function evaluations and thus they may fail to converge in noisy settings. Developing fast and robust quasi-Newton methods for such scenarios is crucial.

To address this issue, we propose a noise-tolerant regularized quasi-Newton method with an Armijo-type line search. The regularization parameter is updated using an Objective-Function-Free Optimization inspired rule that is closely related to the AdaGradNorm method. Under an error model for the objective function and standard assumptions, we establish a global convergence rate of $\mathcal{O}(1/\varepsilon^2)$ for reaching first-order stationary points, matching the optimal rate for first-order methods. Extensive experiments on the CUTEst test collection were performed using 64, 32, and 16-bit floating-point arithmetic and under artificially noisy conditions. The results show that the proposed method is substantially more robust than several existing methods, while maintaining competitive practical convergence speed and computational cost.

Keywords Regularized Quasi-Newton Method, Numerical Stability, Numerical Error, AdaGrad, Objective-Function-Free Optimization

Contents

Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Organization of This Thesis	2
Chapter 2	Foundation of Quasi-Newton Methods	3
2.1	Basic Concepts	3
2.2	Newton's Method	7
2.3	Quasi-Newton Methods	14
2.4	Modified Secant Condition	29
Chapter 3	Noise-Tolerant Regularized Quasi-Newton Method	35
3.1	Problem Setting and Preliminaries	35
3.2	Proposed Algorithm	37
3.3	Theoretical Analysis	39
3.4	Practical Choices of Algorithmic Variables	48
3.5	Numerical Experiments	50
3.6	Conclusion	54
	Acknowledgments	56
A	Appendix	61
A.1	Proof of Lemma 4	61
A.2	Proof of Proposition 16	61

Chapter 1

Introduction

1.1 Introduction

This paper considers the unconstrained optimization problem

$$\underset{x}{\text{minimize}} \quad f(x), \tag{1.1}$$

where $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is a continuously differentiable nonconvex function, and $x \in \mathbb{R}^n$ is the optimization variable. We focus on a noisy optimization setting in which numerical errors contaminate objective function evaluations. Specifically, only inexact function values $\bar{f}(x)$ are available, while gradient information can be computed with comparatively high accuracy. Such inaccuracies are unavoidable in many realistic scenarios, including finite-precision arithmetic, simulation-based evaluations, and stochastic approximations [10, 42, 53]. When function values are contaminated by numerical noise, standard optimization algorithms may lose their theoretical guarantees, behave erratically, or terminate prematurely with unreliable solutions.

In the absence of noise and with accurate function evaluations, quasi-Newton methods, in particular the Broyden–Fletcher–Goldfarb–Shanno (BFGS) method and its limited-memory variant (L-BFGS), are among the most efficient and widely used algorithms to solve Problem (1.1) [37, 44]. Their success is largely attributed to fast local convergence and excellent scalability. Standard implementations [45, 56] rely on line search procedures that enforce the Wolfe conditions to guarantee sufficient descent and the positive definiteness of the Hessian approximation.

In the presence of numerical noise, these line search conditions may become unreliable. Differences in function values or directional derivatives can be dominated by rounding errors, leading to unstable step sizes, ill-conditioned Hessian approximations, or abnormal termination [53]. From a computational perspective, improving the robustness of quasi-Newton methods in noisy environments is therefore crucial. In particular, it is desirable to design algorithms that (i) retain computational efficiency when function evaluations are accurate, (ii) remain stable when function values are unreliable, and (iii) guarantee convergence to first-order stationary points under mild assumptions.

In this work, we propose a noise-tolerant regularized quasi-Newton method that addresses these challenges. Our approach combines Armijo-type line search with quadratic regularization [33, 55] with Objective-Function-Free Optimization (OFFO) inspired strategy [25, 26]. When a sufficient decrease in function value is observed, the method behaves similarly to standard line-search-based quasi-Newton methods, preserving practical efficiency. When numerical noise dominates, we introduce regularization on the approximate Hessian to ensure numerical stability, which yields convergence properties similar to those of OFFO methods, particularly that of ADAGRAD-NORM [58]. The resulting algorithm is particularly well-suited for low-precision or noisy environments.

From a theoretical standpoint, we establish that the proposed method achieves the standard global convergence rate of $\mathcal{O}(1/\varepsilon^2)$ for smooth nonconvex optimization problems, even when objective function values are corrupted by noise. Extensive numerical experiments on the CUTest

benchmark collection, across various floating-point precisions and artificial noise settings, demonstrate that the proposed method is significantly more stable than standard line-search-based methods in settings with significant numerical noise, while maintaining practical convergence speed.

1.2 Organization of This Thesis

The remainder of this thesis is organized as follows.

Chapter 2 reviews the fundamentals of quasi-Newton methods known in the literature. In section 2.1, we introduce basic optimization concepts. In section 2.2, we review Newton’s method for unconstrained optimization. In section 2.3, we present classical quasi-Newton methods with their derivation, mainly focusing on the BFGS and L-BFGS methods. In section 2.4, we discuss the modified secant condition and its variants.

Chapter 3 presents the proposed regularized quasi-Newton method and its theoretical analysis. In section 3.1, we introduce the problem setting and review relevant background. In section 3.2, we present the proposed algorithm. In section 3.3, we establish a global convergence property of the proposed algorithm. In section 3.4, we discuss practical choices of algorithmic variables and parameters. In section 3.5, we report numerical results on CUTEst benchmark problems. Finally, in section 3.6, we conclude the paper with a summary and future research directions.

Our implementation is publicly available on GitHub^{*1}.

^{*1} <https://github.com/HirokiHamaguchi/qnlab>

Chapter 2

Foundation of Quasi-Newton Methods

In this chapter, we review the fundamentals of quasi-Newton methods known in the literature.

2.1 Basic Concepts

One of the central goals of numerical optimization is to determine decision variables that optimize a given quantitative performance measure. Examples include design performance, control stability, operational efficiency, and prediction error, each quantifying the quality of various phenomena or systems. These performance measures are typically modeled by an objective function that maps decision variables to real numbers.

In this chapter, we define $f: \mathbb{R}^n \rightarrow \mathbb{R}$ as an objective function of class C^2 , where n denotes the dimension of the decision variables. We consider the following unconstrained optimization problem:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad f(x). \quad (2.1)$$

In this section, we provide fundamental definitions and properties related to eq. (2.1).

2.1.1 Convexity and Strong Convexity

The notions of convexity and strong convexity are fundamental in optimization theory. Convexity and strong convexity of a function f can be characterized by the following inequalities:

$$(\text{convex}) \quad f(y) \geq f(x) + \nabla f(x)^\top (y - x), \quad (2.2)$$

$$(\mu\text{-strongly convex}) \quad f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|y - x\|^2, \quad (2.3)$$

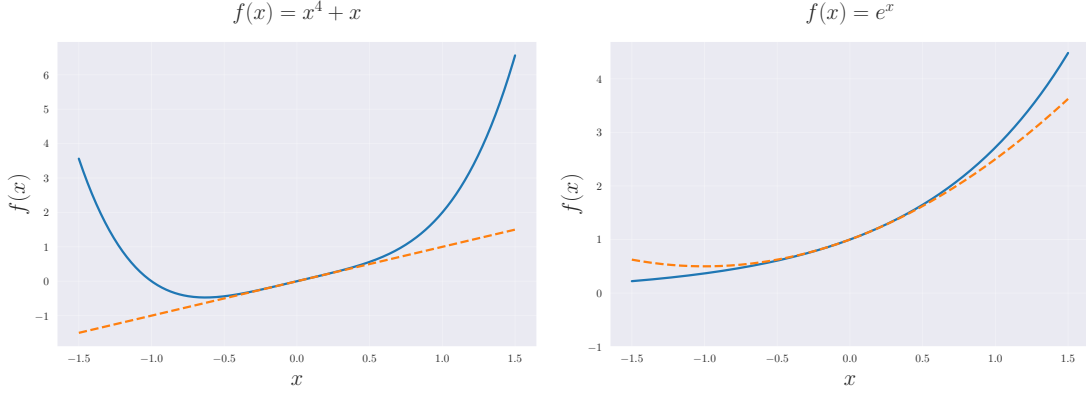
for any $x, y \in \mathbb{R}^n$, where $\mu > 0$ is a constant. Strong convexity indicates that, in addition to convexity, the objective function has uniformly positive curvature. Examples of convex and strongly convex functions are shown in Figure 2.1.

2.1.2 Positive Definiteness of the Hessian

Next, we show how convexity and strong convexity relate to the definiteness of the Hessian matrix $\nabla^2 f(x)$. Let A be a symmetric matrix in $\mathbb{R}^{n \times n}$. A matrix A is called positive or negative definite (or semi-definite) based on the following conditions:

$$\begin{aligned} (\text{positive definite}) \quad & v^\top A v > 0 \quad \forall v \in \mathbb{R}^n \setminus \{0\}, \\ (\text{positive semi-definite}) \quad & v^\top A v \geq 0 \quad \forall v \in \mathbb{R}^n, \\ (\text{negative definite}) \quad & v^\top A v < 0 \quad \forall v \in \mathbb{R}^n \setminus \{0\}, \end{aligned}$$

Convex but not strongly convex



Strongly convex

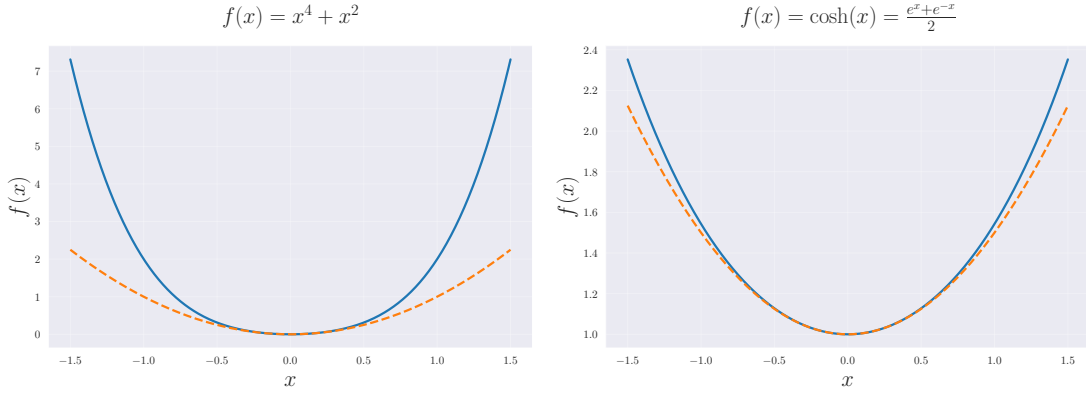


Fig. 2.1: Comparison of convex and strongly convex functions. The dashed line shows the quadratic approximation at $x = 0$. The upper two functions are convex but not strongly convex, and no constant $\mu > 0$ satisfies eq. (2.3). The lower two functions are strongly convex, and there exists $\mu > 0$ that satisfies eq. (2.3).

$$(\text{negative semi-definite}) \quad v^\top A v \leq 0 \quad \forall v \in \mathbb{R}^n.$$

A matrix is indefinite if it is neither positive nor negative definite. For matrices $A, B \in \mathbb{R}^{n \times n}$, the notation $A \succeq B$ indicates that $A - B$ is positive semi-definite. For $\mu > 0$, the condition $A \succeq \mu I$ is equivalent to $v^\top A v \geq \mu \|v\|^2$ for all $v \in \mathbb{R}^n$. It directly implies that all eigenvalues of A are at least μ , which in turn implies that the operator norm $\|A\|$ is at least μ , i.e., $\|A\| \geq \mu$. In particular, if B is a zero matrix, we simply write $A \succeq 0$. We similarly define $\preceq$ for negative semi-definiteness.

We can relate convexity and strong convexity to the definiteness of the Hessian as follows.

Proposition 1. *Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^2 . Then*

- *f is convex if and only if $\nabla^2 f(x) \succeq 0$ holds for all $x \in \mathbb{R}^n$.*
- *f is μ -strongly convex if and only if $\nabla^2 f(x) \succeq \mu I$ holds for all $x \in \mathbb{R}^n$.*

Proof. First, let $\mu > 0$ and assume $\nabla^2 f(x) \succeq \mu I$ for all $x \in \mathbb{R}^n$. By the fundamental theorem of calculus, for any $x, y \in \mathbb{R}^n$, we have

$$f(y) = f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2} \int_0^1 (y - x)^\top \nabla^2 f(x + t(y - x)) (y - x) dt. \quad (2.4)$$

Positive definite Indefinite Negative definite

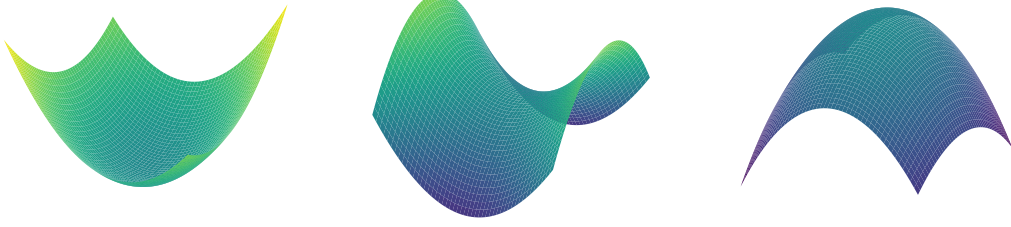


Fig. 2.2: Quadratic model $f(x) = \frac{1}{2}(x - x_k)^\top H(x - x_k) + \nabla f(x_k)^\top (x - x_k) + f(x_k)$ in 2-dimensional space with Hessians H which is (left) positive definite, (center) indefinite, (right) negative definite.

Then, we obtain

$$\int_0^1 (y - x)^\top \nabla^2 f(x + t(y - x))(y - x) dt \geq \int_0^1 \mu \|y - x\|^2 dt = \mu \|y - x\|^2. \quad (2.5)$$

Thus, substituting eq. (2.5) into eq. (2.4) yields eq. (2.3), the definition of μ -strong convexity.

Conversely, if f is μ -strongly convex, for any $x \in \mathbb{R}^n$, $v \in \mathbb{R}^n$ and $t > 0$, letting $y = x \pm tv$ gives

$$\begin{cases} f(x + tv) \geq f(x) + t \nabla f(x)^\top v + \frac{\mu}{2} t^2 \|v\|^2, \\ f(x - tv) \geq f(x) - t \nabla f(x)^\top v + \frac{\mu}{2} t^2 \|v\|^2. \end{cases} \quad (2.6)$$

By Taylor's theorem, there exists $s_\pm \in (0, 1)$ such that

$$\begin{cases} f(x + tv) = f(x) + t \nabla f(x)^\top v + \frac{1}{2} t^2 v^\top \nabla^2 f(x + s_+ tv) v, \\ f(x - tv) = f(x) - t \nabla f(x)^\top v + \frac{1}{2} t^2 v^\top \nabla^2 f(x - s_- tv) v. \end{cases} \quad (2.7)$$

Substituting eq. (2.7) into eq. (2.6) and rearranging yields

$$v^\top \frac{\nabla^2 f(x + s_+ tv) + \nabla^2 f(x - s_- tv)}{2} v \geq \mu \|v\|^2. \quad (2.8)$$

Letting $t \rightarrow 0$ in eq. (2.8) and using the continuity of $\nabla^2 f$ from the C^2 assumption gives

$$v^\top \nabla^2 f(x) v \geq \mu \|v\|^2.$$

Because $v \in \mathbb{R}^n$ is arbitrary, we obtain $\nabla^2 f(x) \succeq \mu I$.

Setting $\mu = 0$ in the above shows the convex case corresponding to eq. (2.2) in the same way. $\square$

Quadratic functions whose Hessians are positive definite, indefinite, and negative definite are illustrated in Figure 2.2. One can visually confirm the correspondence between positive definiteness and convexity.

2.1.3 L -smoothness

Finally, we introduce L -smoothness of functions. A function f is L -smooth if

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad (2.9)$$

holds for all x, y .

The next proposition shows that L -smoothness can be characterized by the upper bound of the Hessian.

Proposition 2. *Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^2 . Then f is L -smooth if and only if $\nabla^2 f(x) \preceq LI$ holds for all $x \in \mathbb{R}^n$.*

Proof. Since f is of class C^2 , by the fundamental theorem of calculus, for any $x, y \in \mathbb{R}^n$, we have

$$\nabla f(y) - \nabla f(x) = \int_0^1 \nabla^2 f(x + t(y-x))(y-x) dt. \quad (2.10)$$

Assume $\nabla^2 f(x) \preceq LI$ for all $x \in \mathbb{R}^n$. It implies that the operator norm of $\nabla^2 f(x)$ satisfies $\|\nabla^2 f(x)\| \leq L$, and thus

$$\begin{aligned} \|\nabla f(y) - \nabla f(x)\| &= \left\| \int_0^1 \nabla^2 f(x + t(y-x))(y-x) dt \right\| && \text{(by eq. (2.10))} \\ &\leq \int_0^1 \|\nabla^2 f(x + t(y-x))(y-x)\| dt && \text{(triangle inequality)} \\ &\leq \int_0^1 L\|y-x\| dt && \text{(by assumption)} \\ &= L\|y-x\|, \end{aligned}$$

which shows eq. (2.9). Conversely, if f is L -smooth, then for any $x \in \mathbb{R}^n$ and $v \in \mathbb{R}^n$, we have

$$\|\nabla f(x+tv) - \nabla f(x)\| \leq L\|tv\| = Lt\|v\| \quad (2.11)$$

By Taylor's theorem, we also have

$$\nabla f(x+tv) - \nabla f(x) = t\nabla^2 f(x)v + r(t),$$

where $r(t)$ satisfies $\|r(t)\|/t \rightarrow 0$ as $t \rightarrow 0$, and we can rewrite it as

$$\nabla^2 f(x)v = \lim_{t \rightarrow 0} \frac{\nabla f(x+tv) - \nabla f(x) - r(t)}{t} = \lim_{t \rightarrow 0} \frac{\nabla f(x+tv) - \nabla f(x)}{t}. \quad (2.12)$$

Taking the inner product with v gives

$$\begin{aligned} v^\top \nabla^2 f(x)v &= \lim_{t \rightarrow 0} \left(\frac{\nabla f(x+tv) - \nabla f(x)}{t} \right)^\top v \\ &\leq \lim_{t \rightarrow 0} \frac{\|\nabla f(x+tv) - \nabla f(x)\|}{t} \|v\| && \text{(Cauchy-Schwarz inequality)} \\ &\leq L\|v\|^2. && \text{(by eq. (2.11))} \end{aligned}$$

Because $v \in \mathbb{R}^n$ is arbitrary, we obtain $\nabla^2 f(x) \preceq LI$. □

Baillon–Haddad Theorem

One of the useful properties of L -smooth functions is given by the following Baillon–Haddad theorem. Note that only C^1 differentiability is required here.

Proposition 3 (Baillon–Haddad theorem). *Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^1 . If f is L -smooth and convex, then for all $x, y \in \mathbb{R}^n$, ∇f is $1/L$ -cocoercive, i.e.,*

$$(\nabla f(x) - \nabla f(y))^\top (x - y) \geq \frac{1}{L} \|\nabla f(x) - \nabla f(y)\|^2 \quad (2.13)$$

for all $x, y \in \mathbb{R}^n$.

We refer to other literature for the proof [5] [49, Proposition 12.60]. One consequence of this theorem is that, for the sequences $\{x_k\}$ generated by an optimization algorithm, defining

$$s_k := x_{k+1} - x_k, \quad y_k := \nabla f(x_{k+1}) - \nabla f(x_k)$$

and applying eq. (2.13) with $x = x_{k+1}$ and $y = x_k$ yields

$$s_k^\top y_k \geq \frac{1}{L} \|y_k\|^2.$$

This inequality is sometimes used in the analysis of update formulas in methods such as BFGS and L-BFGS, and we will also use it in a later chapter.

Bound on Hessian Eigenvalues

The L -smoothness combined with μ -strong convexity also provides bounds on the eigenvalues of the Hessian.

Proposition 4. *Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^2 . If f is L -smooth and μ -strongly convex, then the eigenvalues of the Hessian $\nabla^2 f(x)$ are contained in the interval $[\mu, L]$ for all $x \in \mathbb{R}^n$.*

Proof. By Propositions 1 and 2, we have

$$\mu I \preceq \nabla^2 f(x) \preceq LI,$$

which directly implies that the eigenvalues of $\nabla^2 f(x)$ are contained in the interval $[\mu, L]$. $\square$ $\square$

As shown in Proposition 4, the L -smoothness and μ -strong convexity impose upper and lower bounds on the eigenvalues of the Hessian, respectively.

2.2 Newton's Method

2.2.1 Algorithm of Newton's Method

In this subsection, we outline Newton's method, which is a fundamental optimization algorithm for an unconstrained optimization problem. Newton's method is a representative iterative algorithm that starts from an initial point x_0 , successively updates the current point, and generates a sequence $\{x_k\}_{k=0}^\infty$.

We assume that f is of class C^2 and strongly convex, so that the Hessian matrix $\nabla^2 f(x)$ is positive definite for all $x \in \mathbb{R}^n$, and thus invertible. Using the gradient $g_k := \nabla f(x_k)$ and Hessian $\nabla^2 f(x_k)$ at the k -th iterate x_k , the base procedure of Newton's method is to iteratively update

$$x_{k+1} \leftarrow x_k - \alpha_k \nabla^2 f(x_k)^{-1} g_k$$

with a step size $\alpha_k > 0$ determined by line search. The derivation of this update rule is as follows. The quadratic Taylor approximation of f around x_k is given by

$$m_k^*(x) := f(x_k) + g_k^\top (x - x_k) + \frac{1}{2} (x - x_k)^\top \nabla^2 f(x_k) (x - x_k). \quad (2.14)$$

The gradient of this model is

$$\nabla m_k^*(x) = g_k + \nabla^2 f(x_k) (x - x_k). \quad (2.15)$$

Since the quadratic model eq. (2.14) is strongly convex by assumption, $x^* \in \mathbb{R}^n$ is the minimizer of m_k^* if and only if $\nabla m_k^*(x) = 0$, so solving $\nabla^2 f(x_k) (x - x_k) = -g_k$ yields

$$\arg \min_{x \in \mathbb{R}^n} m_k^*(x) = x_k - \nabla^2 f(x_k)^{-1} g_k.$$

Although this choice appears natural, it does not in general guarantee global convergence, as we will see in section 2.2.3. Thus, we introduce a step size $\alpha_k > 0$ determined by line search to ensure a sufficient decrease in the function value. This is the basic procedure of Newton's method.

The update direction $d_k := -\nabla^2 f(x_k)^{-1} g_k$ is known as the Newton direction, which is a descent direction when the Hessian is positive definite, since

$$g_k^\top d_k = -g_k^\top \nabla^2 f(x_k)^{-1} g_k < 0.$$

If f is nonconvex and the Hessian $\nabla^2 f(x_k)$ is negative definite or indefinite, function values are not guaranteed to decrease, and Newton's method may fail to converge. Therefore, checking the positive definiteness of the Hessian is important when applying Newton's method. The modified Newton method [44, Sec. 3.4] is one remedy for handling such cases.

2.2.2 Properties Related to Convergence

We next present a standard convergence theorem for Newton's method.

Global Convergence

We first state a general global convergence result

$$\lim_{k \rightarrow \infty} \|g_k\| = 0$$

for methods with line search, and then explain its application to Newton's method. We consider a general iterative method of the form

$$x_{k+1} \leftarrow x_k + \alpha_k d_k,$$

where d_k is a descent direction satisfying $g_k^\top d_k < 0$ and $\alpha_k > 0$ is the step size determined by line search. We employ the Wolfe conditions [44, Sec. 3.1] to determine the step size α_k as follows:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k g_k^\top d_k, \quad (2.16)$$

$$g_{k+1}^\top d_k \geq c_2 g_k^\top d_k, \quad (2.17)$$

where $0 < c_1 < c_2 < 1$ are constants. We also define the angle between the direction d_k and the negative gradient $-g_k$ as

$$\cos \theta_k := \frac{-g_k^\top d_k}{\|g_k\| \|d_k\|}. \quad (2.18)$$

The following theorem is a simplified version of the classical result.

Theorem 1 ([44, Theorem 3.2]). Suppose that f is of class C^1 and bounded below in $\mathbb{R}^n$, and L -smooth. Consider the iterative method defined by

$$x_{k+1} \leftarrow x_k + \alpha_k d_k,$$

starting from an initial point $x_0 \in \mathbb{R}^n$, and the step size α_k is determined by the Wolfe conditions eqs. (2.16) and (2.17). For the angle θ_k defined in eq. (2.18), if there exists a positive constant δ such that $\cos \theta_k \geq \delta > 0$ for all k , then, the method generates a sequence $\{x_k\}$ satisfying

$$\lim_{k \rightarrow \infty} \|g_k\| = 0. \quad (2.19)$$

Proof. From the Wolfe condition (2.17), we have

$$(g_{k+1} - g_k)^\top d_k = g_{k+1}^\top d_k - g_k^\top d_k \geq (c_2 - 1)g_k^\top d_k,$$

and

$$\begin{aligned} (g_{k+1} - g_k)^\top d_k &\leq \|g_{k+1} - g_k\| \|d_k\| && \text{(Cauchy-Schwarz inequality)} \\ &\leq L \|x_{k+1} - x_k\| \|d_k\| && \text{(L-smoothness)} \\ &= \alpha_k L \|d_k\|^2. \end{aligned}$$

By combining these two relations, we obtain

$$\alpha_k \geq \frac{c_2 - 1}{L} \frac{g_k^\top d_k}{\|d_k\|^2}. \quad (2.20)$$

and thus

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) + c_1 \alpha_k g_k^\top d_k && \text{(Wolfe condition (2.16))} \\ &\leq f(x_k) - c_1 \frac{1 - c_2}{L} \frac{(g_k^\top d_k)^2}{\|d_k\|^2} && \text{(by (2.20))} \\ &= f(x_k) - c_1 \frac{1 - c_2}{L} \cos^2 \theta_k \|g_k\|^2. && \text{(definition in (2.18))} \end{aligned}$$

By summing this expression over all indices less than or equal to k , we obtain

$$f(x_{k+1}) \leq f(x_0) - c_1 \frac{1 - c_2}{L} \sum_{j=0}^k \cos^2 \theta_j \|g_j\|^2. \quad (2.21)$$

Since f is bounded below, we have that $f(x_0) - f(x_{k+1})$ is less than some positive constant for all k . Hence, by taking limits in (2.21), we obtain the Zoutendijk condition:

$$\sum_{k=0}^{\infty} \cos^2 \theta_k \|g_k\|^2 < \infty. \quad (2.22)$$

This condition implies that

$$\cos^2 \theta_k \|g_k\|^2 \rightarrow 0.$$

Combined with the assumption that $\cos \theta_k \geq \delta > 0$ for all k , it follows immediately that

$$\lim_{k \rightarrow \infty} \|g_k\| = 0,$$

which completes the proof. $\square$

Since Newton's method is a special case of the setting in Theorem 1 with $d_k = -\nabla^2 f(x_k)^{-1} g_k$, we can apply this result under appropriate conditions of f . In particular, if f is L -smooth and μ -strongly convex, then for any k , we have

$$\cos \theta_k = \frac{g_k^\top \nabla^2 f(x_k)^{-1} g_k}{\|g_k\| \|\nabla^2 f(x_k)^{-1} g_k\|} \geq \frac{\|g_k\|^2 \lambda_{\min}(\nabla^2 f(x_k)^{-1})}{\|g_k\|^2 \lambda_{\max}(\nabla^2 f(x_k)^{-1})} \geq \frac{\mu}{L},$$

where $\lambda_{\min}(\cdot)$ and $\lambda_{\max}(\cdot)$ denote the minimum and maximum eigenvalues, respectively. Thus, by setting $\delta = \mu/L$ in Theorem 1, we obtain the global convergence of Newton's method for strongly convex and smooth functions under line search satisfying the Wolfe conditions.

Local Quadratic Convergence

We next present a classical result on the local convergence rate of Newton's method. We again provide a simplified version for brevity as follows.

Theorem 2 ([44, Theorem 3.5]). *Suppose that the Hessian matrix $\nabla^2 f(x)$ is Lipschitz continuous in a neighborhood of the solution x^* , and that sufficient second-order optimality conditions hold (i.e., $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*)$ is positive definite). If $\alpha_k = 1$ for all k and the initial point x_0 is sufficiently close to x^* , then the sequence of gradient norms $\{\|\nabla f(x_k)\|\}$ converges quadratically.*

Proof. From the definition of the Newton step and the optimality condition $\nabla f(x^*) = 0$, we obtain

$$\begin{aligned} x_{k+1} - x^* &= x_k - x^* - (\nabla^2 f(x_k))^{-1} \nabla f(x_k) \\ &= (\nabla^2 f(x_k))^{-1} (\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))). \end{aligned}$$

By Taylor's theorem and the triangle inequality, we have

$$\begin{aligned} &\|\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))\| \\ &= \left\| \nabla^2 f(x_k)(x_k - x^*) - \int_0^1 \nabla^2 f(x_k + t(x^* - x_k))(x_k - x^*) dt \right\| \\ &\leq \int_0^1 \|\nabla^2 f(x_k) - \nabla^2 f(x_k + t(x^* - x_k))\| \|x_k - x^*\| dt. \end{aligned}$$

If $\nabla^2 f$ is Lipschitz continuous with constant L^H , then the integrand is bounded by $L^H t \|x_k - x^*\|$, and integrating yields

$$\|\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))\| \leq \frac{1}{2} L^H \|x_k - x^*\|^2.$$

Since $\nabla^2 f(x^*)$ is non-singular, there exists a radius $r > 0$ such that for all x_k satisfying $\|x_k - x^*\| \leq r$,

$$\|(\nabla^2 f(x_k))^{-1}\| \leq 2 \|(\nabla^2 f(x^*))^{-1}\|$$

holds. Combining these results, we obtain

$$\|x_{k+1} - x^*\| \leq L^H \|(\nabla^2 f(x^*))^{-1}\| \|x_k - x^*\|^2.$$

Let $\tilde{L} := L^H \|(\nabla^2 f(x^*))^{-1}\|$. If the initial point is chosen such that $\|x_0 - x^*\| \leq \min\{r, 1/(2\tilde{L})\}$, then by induction $\{x_k\}$ remains within the neighborhood and converges to x^* . The above error bound implies quadratic convergence of $\{x_k\}$.

To show quadratic convergence of the gradient norm, we use $x_{k+1} - x_k = -(\nabla^2 f(x_k))^{-1} \nabla f(x_k)$ and $\nabla f(x_k) + \nabla^2 f(x_k)(x_{k+1} - x_k) = 0$, yielding

$$\begin{aligned} \|\nabla f(x_{k+1})\| &= \|\nabla f(x_{k+1}) - \nabla f(x_k) - \nabla^2 f(x_k)(x_{k+1} - x_k)\| \\ &\leq \int_0^1 \|\nabla^2 f(x_k + t(x_{k+1} - x_k)) - \nabla^2 f(x_k)\| \|x_{k+1} - x_k\| dt \\ &\leq \frac{1}{2} L^H \|x_{k+1} - x_k\|^2 \\ &\leq \frac{1}{2} L^H \left\| (\nabla^2 f(x_k))^{-1} \right\|^2 \|\nabla f(x_k)\|^2 \\ &\leq 2L^H \left\| (\nabla^2 f(x^*))^{-1} \right\|^2 \|\nabla f(x_k)\|^2. \end{aligned}$$

Therefore, $\|\nabla f(x_k)\|$ converges quadratically to zero. $\square$

These propositions Theorems 1 and 2 indicate that Newton's method equipped with line search has both global convergence and local quadratic convergence properties under appropriate conditions. This rapid convergence is a significant advantage of Newton's method compared to first-order methods such as gradient descent, which typically exhibit only linear convergence. However, computing the Hessian $\nabla^2 f(x_k) \in \mathbb{R}^{n \times n}$ and solving the linear system $\nabla^2 f(x_k)d_k = -\nabla f(x_k)$ require $\mathcal{O}(n^3)$ time, which is prohibitively expensive for large-scale problems. Other optimization methods, such as quasi-Newton methods, are also used for such methods, as we will discuss later.

2.2.3 Elements for Global Convergence

As mentioned earlier, the positive definiteness of the Hessian matrix and the selection of the step size via line search play important roles in Newton's method. In this subsection, we explain why these elements are required.

Positive Definiteness of the Hessian Matrix

We have assumed that f is strongly convex so far, meaning that the Hessian matrix $\nabla^2 f(x_k)$ is positive definite at each iteration. This assumption is crucial for Newton's method to converge locally to an optimal solution. When the Hessian matrix is positive definite, Newton's method provides a descent direction toward the optimal solution, since

$$\nabla f(x_k)^\top d_k = -\nabla f(x_k)^\top (\nabla^2 f(x_k))^{-1} \nabla f(x_k) < 0.$$

In contrast, when the Hessian matrix is negative definite or indefinite, a decrease in the function value is not guaranteed, and Newton's method may point in a direction that moves away from the optimal solution. In the indefinite case, the function value may decrease, but the risk of converging to a saddle point is also increased. Therefore, verifying the positive definiteness of the Hessian matrix is an important consideration when applying Newton's method.

Issues with Full Steps

In Newton's method, adopting a step size $\alpha_k = 1$ at every iteration may lead to difficulties. Here, we present examples illustrating this issue and discuss the necessity of line search as a remedy.

An Example Where Newton's Method Diverges Consider the following function:

$$\begin{aligned} f(x) &= \sqrt{1+x^2}, \\ f'(x) &= \frac{x}{\sqrt{1+x^2}}, \end{aligned}$$

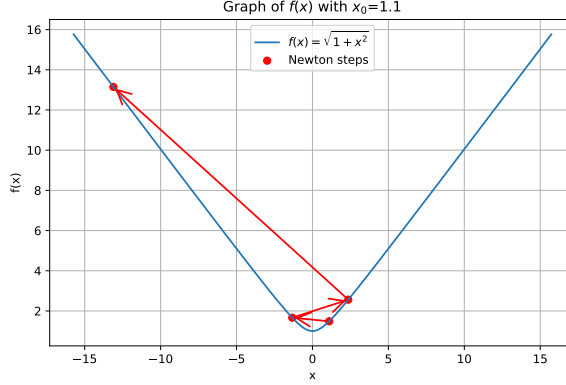


Fig. 2.3: An example where Newton's method diverges with initial point $x_0 = 1.1$

$$f''(x) = \frac{1}{(1+x^2)^{3/2}}.$$

When the absolute value of the initial point exceeds 1, Newton's method diverges, as shown in Figure 2.3.

Oscillation of Newton's Method for Strongly Convex Functions In the previous example, the objective function was not strongly convex and did not necessarily possess favorable properties. However, even for functions with the strong convexity property, there exist examples where Newton's method fails to converge [15, Example 1.4.3].

In this example, for $\mu > 0$, consider the function

$$\begin{aligned} f(x) &= \log(1 + e^x) - \frac{x}{2} + \frac{\mu x^2}{2}, \\ f'(x) &= \frac{e^x}{1 + e^x} - \frac{1}{2} + \mu x, \\ f''(x) &= \frac{e^x}{(1 + e^x)^2} + \mu, \\ f'''(x) &= \frac{e^x(1 - e^x)}{(1 + e^x)^3}, \\ f''''(x) &= \frac{e^x(1 - 4e^x + e^{2x})}{(1 + e^x)^4}. \end{aligned}$$

This function has the following properties:

- It is μ -strongly convex.
- $\max_x |f''(x)| = \frac{1}{4} + \mu$ (attained at $e^x = 1$). Hence, ∇f is L -smooth with $L = \frac{1}{4} + \mu$.
- $\max_x |f'''(x)| = \frac{1}{6\sqrt{3}}$ (attained at $e^x = 2 - \sqrt{3}$). Hence, $\nabla^2 f$ is M -Lipschitz with $M = \frac{1}{6\sqrt{3}}$.

Nevertheless, when the initial point x_0 is sufficiently large relative to μ , Newton's method exhibits oscillatory behavior, as shown in Figure 2.4.

Necessity of Line Search To avoid the issues described above, it is common practice to select the step size α_k appropriately using a line search. Newton's method equipped with line search is often referred to as the modified Newton method and is known to possess global convergence properties.

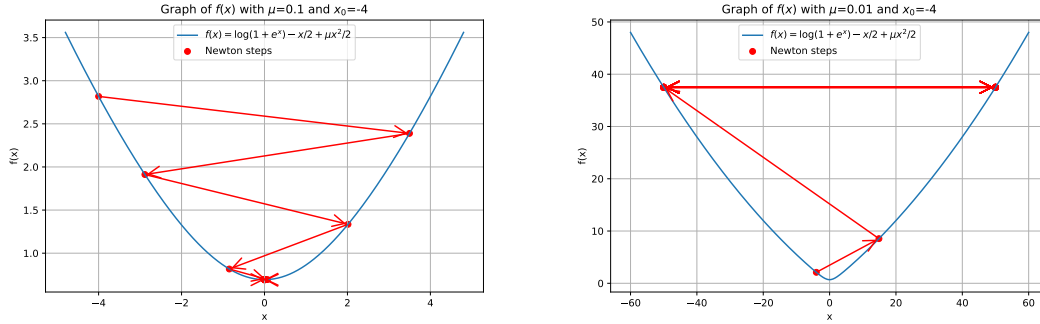


Fig. 2.4: (Left) Newton's method converges for $x_0 = -4$, $\mu = 0.1$; (Right) Newton's method oscillates for $x_0 = -4$, $\mu = 0.01$

2.2.4 Comparison with Newton's Method as a Root-Finding Algorithm

As a conceptual supplement, we briefly clarify the relationship between “Newton's method as a root-finding algorithm” and “Newton's method in optimization.” These two formulations are closely related, but they have different perspectives and applications.

Newton's Method as a Root-Finding Algorithm

When referring simply to Newton's method (or the Newton–Raphson method), one often means a root-finding algorithm for a differentiable scalar function $g: \mathbb{R} \rightarrow \mathbb{R}$ that solves $g(x) = 0$. Although this is not the main topic of this section, it is arguably more fundamental and widely known.

Starting from an initial value x_0 , the iteration is given by

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)}.$$

Geometrically, this corresponds to approximating the graph of g near x_k by its tangent line and selecting the intersection of this tangent with the x -axis as the next approximate solution.

Newton's Method in Optimization

In the context of optimization, Newton's method typically refers to an algorithm for finding a local minimizer of a twice-differentiable function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, or equivalently, a stationary point satisfying $\nabla f(x) = 0$ as a necessary condition. This is the primary focus of the present section.

Recall the assumption that the Hessian matrix $\nabla^2 f(x)$ is positive definite at each iteration. Newton's method in optimization applies the framework of the root-finding algorithm to the gradient $\nabla f(x)$:

$$x_{k+1} = x_k - \nabla^2 f(x_k)^{-1} \nabla f(x_k).$$

In the scalar case, this reduces to $x_{k+1} = x_k - f'(x_k)/f''(x_k)$.

Relationship Between the Two Formulations

Let us examine the equivalence of the two formulations through a concrete example. Figure 2.5 illustrates the relationship between these formulations using a simple cubic function $f(x)$. The left panel shows root-finding applied to $g(x) = \nabla f(x)$, while the right panel shows optimization applied to $f(x)$.

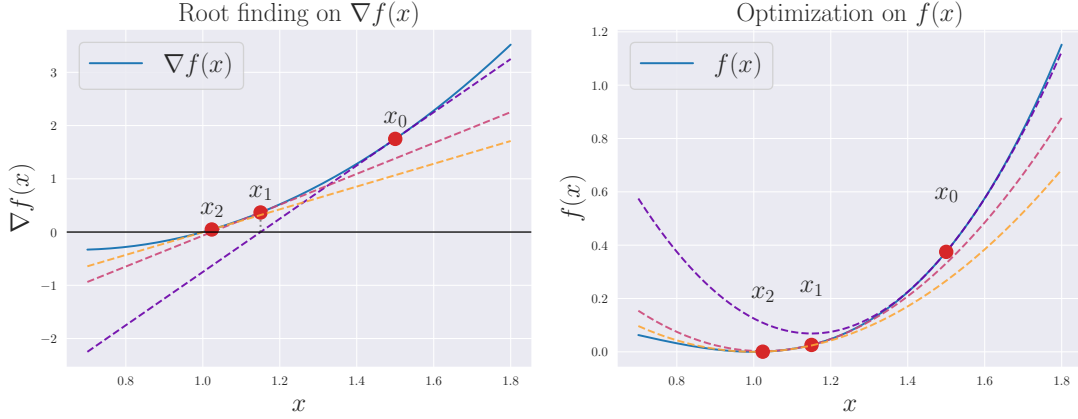


Fig. 2.5: Root-finding for the gradient $\nabla f(x) = 3x^2 - 4x + 1$ and optimization of the function $f(x) = x^3 - 2x^2 + x$

In the root-finding formulation, $\nabla f(x) = 3x^2 - 4x + 1$ is approximated by its tangent line, namely

$$\nabla m_k^*(x) = \nabla f(x_k) + \nabla^2 f(x_k)(x - x_k),$$

and the root of this linear model is chosen as x_{k+1} .

In contrast, in the optimization formulation, $f(x) = x^3 - 2x^2 + x$ is approximated by its second-order Taylor expansion

$$m_k^*(x) = f(x_k) + \nabla f(x_k)(x - x_k) + \frac{1}{2} \nabla^2 f(x_k)(x - x_k)^2,$$

and the minimizer of this quadratic model is taken as x_{k+1} .

Thus, under the assumption of positive definiteness, it is clear from the properties of local solutions that both formulations perform essentially equivalent operations, and their correspondence can be explicitly verified.

In what follows, we consider only the optimization formulation.

2.3 Quasi-Newton Methods

In this section, we discuss quasi-Newton methods. Quasi-Newton methods are based on Newton's method, but aim to reduce its primary drawback, namely, the high computational cost of evaluating the Hessian matrix. Specifically, instead of using the true Hessian matrix $\nabla^2 f(x_k)$, an approximation matrix B_k is employed, thereby achieving fast convergence while keeping the computational cost low.

Figure 2.6 shows a comparison of Newton's method, quasi-Newton methods, and gradient descent. Gradient descent is a method that simply updates in the direction opposite to the gradient. Although its per-iteration computational cost is the lowest, its convergence is slow. Newton's method converges in the fewest iterations, but has the highest computational cost per step. Quasi-Newton methods lie between these two extremes, striking a balance between them. In particular, when comparing methods in terms of computation time rather than the number of iterations, quasi-Newton methods often exhibit the best performance.

Line-search-based quasi-Newton methods generate a sequence $\{x_k\}_{k=0}^{\infty}$ converging to the optimal solution x^* as follows:

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k) = x_k - \alpha_k H_k \nabla f(x_k)$$

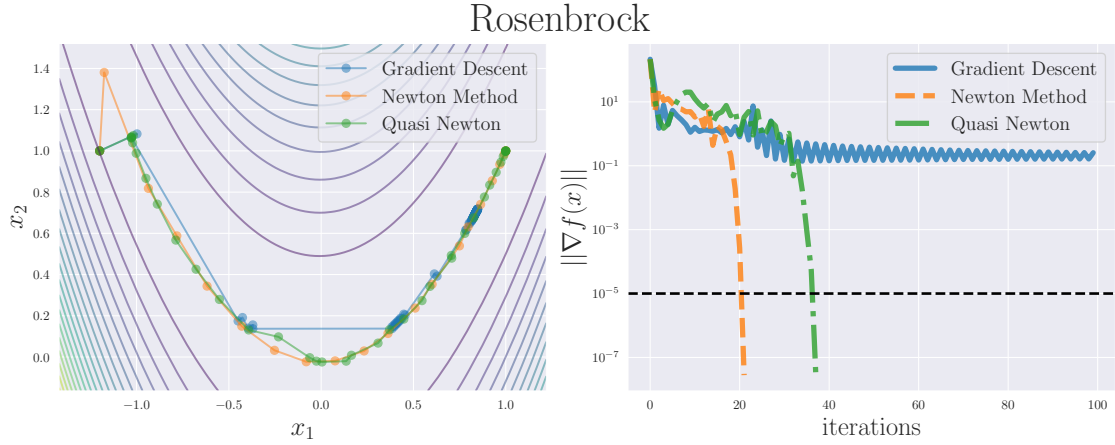


Fig. 2.6: Comparison of Newton's method, quasi-Newton methods, and gradient descent

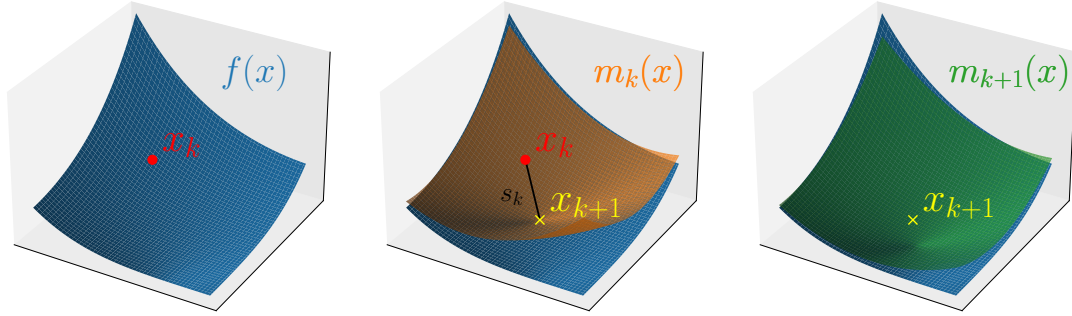


Fig. 2.7: Conceptual illustration of quasi-Newton methods. (1) The objective function f (blue surface) and the current point x_k (red dot). (2) The quadratic model induced by the current Hessian approximation (orange surface) and its minimizer x_{k+1} (yellow cross). (3) The updated quadratic model based on the new point x_{k+1} (green surface).

Here, $\alpha_k > 0$ is the step size determined by line search, B_k is an approximation of the Hessian matrix $\nabla^2 f(x_k)$ at the point x_k , and $H_k := B_k^{-1}$ denotes its inverse.

The core of quasi-Newton methods lies in how to update B_k (or its inverse H_k) at each iteration so that it approaches the Hessian matrix $\nabla^2 f(x_k)$. Figure 2.7 illustrates this concept. First, around the current point x_k , a quadratic approximation model of the objective function f is constructed using B_k . Next, this quadratic model is minimized to obtain the next point x_{k+1} . After obtaining x_{k+1} , the approximation matrix B_k is updated to B_{k+1} using the gradient information at x_k and x_{k+1} . This procedure is repeated until convergence, which constitutes the quasi-Newton method.

In what follows, we introduce the secant condition that the approximate Hessian matrix B_k is required to satisfy, and present several representative update formulas for such B_k and H_k .

2.3.1 Secant Condition

In this section as well, let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be of class C^2 . Suppose that a symmetric matrix B_k is given as an approximation of the Hessian matrix $\nabla^2 f(x_k)$. We determine an approximation Hessian B_{k+1} at the next point x_{k+1} . Although there are infinitely many possible candidates for such a matrix, it is natural to impose symmetry on B_{k+1} as well, given that the true Hessian matrix is

symmetric. Define the step and gradient difference as

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k),$$

where we assume that $y_k^\top s_k \neq 0$ and $s_k \neq 0$. Here, the s in s_k stands for step. Using the Taylor expansion of $\nabla f(x_k)$, we have

$$\begin{aligned} \nabla f(x_k) &= \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}) + \mathcal{O}(\|x_k - x_{k+1}\|^2) \\ &\approx \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}) \\ &\approx \nabla f(x_{k+1}) + B_{k+1}(x_k - x_{k+1}) \end{aligned}$$

Requiring the above approximation to hold as an equality yields

$$B_{k+1}(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k),$$

or equivalently,

$$B_{k+1}s_k = y_k. \quad (2.23)$$

The relation in eq. (2.23) is called the secant condition, or the quasi-Newton equation.

2.3.2 Representative Quasi-Newton Update Rules

Given B_k , s_k , and y_k , there exist many update rules that produce B_{k+1} satisfying the secant condition. We introduce several representative ones with their derivations [14]. In this subsection only, for brevity, we abbreviate B_k , B_{k+1} , s_k , and y_k as B , $\bar{B}$, s , and y , respectively.

Broyden's Update

Broyden's update is one of the most fundamental quasi-Newton update formulas, but it is not popular in practice due to its lack of symmetry preservation. The update formulas are given by

$$\bar{B}_{\text{Broyden}} = B + \frac{(y - Bs)s^\top}{s^\top s}, \quad (2.24)$$

$$\bar{H}_{\text{Broyden}} = H + \frac{s - Hy}{s^\top Hy} s^\top H. \quad (2.25)$$

Derivation We derive this formula from simple structural assumptions [14, Section 4].

Proposition 5. Assume that $\bar{B}$ satisfies the secant condition

$$\bar{B}s = y, \quad (2.26)$$

and the action constraint

$$\bar{B}z = Bz \quad \text{for all } z \in \mathbb{R}^n \text{ such that } z^\top s = 0. \quad (2.27)$$

Then $\bar{B}$ is uniquely determined and it is $\bar{B}_{\text{Broyden}}$ defined by eq. (2.24).

Proof. A basis for $\mathbb{R}^n$ can be constructed from s and a basis for the orthogonal complement of s . Since eqs. (2.26) and (2.27) completely determine the action of $\bar{B}$ with respect to this basis, $\bar{B}$ is uniquely determined.

We now show that $\bar{B}_{\text{Broyden}}$ defined by eq. (2.24) satisfies eqs. (2.26) and (2.27). Let z be any vector satisfying $z^\top s = 0$. Then

$$\bar{B}_{\text{Broyden}}s = \left(B + \frac{(y - Bs)s^\top}{s^\top s} \right) s = Bs + (y - Bs) = y,$$

$$\bar{B}_{\text{Broyden}} z = \left(B + \frac{(y - Bs)s^\top}{s^\top s} \right) z = Bz + (z^\top s) \frac{y - Bs}{s^\top s} = Bz.$$

Hence $\bar{B}_{\text{Broyden}}$ indeed satisfies eqs. (2.26) and (2.27). Therefore, by uniqueness, we have $\bar{B} = \bar{B}_{\text{Broyden}}$. $\square$

Broyden's update can also be characterized as a minimal-change update in the Frobenius norm.

Proposition 6 ([14], Theorem 4.1). *Let $B \in \mathbb{R}^{n \times n}$, $y \in \mathbb{R}^n$, and $s \in \mathbb{R}^n \setminus \{0\}$ be given. Then the matrix $\bar{B}$ defined by eq. (2.24) is the unique solution of*

$$\begin{aligned} & \underset{\tilde{B} \in \mathbb{R}^{n \times n}}{\text{minimize}} && \|\tilde{B} - B\|_F \\ & \text{subject to} && \tilde{B}s = y. \end{aligned}$$

Proof. The function $\tilde{B} \mapsto \|\tilde{B} - B\|_F$ is strictly convex on $\mathbb{R}^{n \times n}$. The constraint set

$$\{\tilde{B} \in \mathbb{R}^{n \times n} : \tilde{B}s = y\} \quad (2.28)$$

is affine and hence convex. Therefore, the optimization problem admits at most one minimizer.

To show that $\bar{B} = \bar{B}_{\text{Broyden}}$ defined by eq. (2.24) is indeed the minimizer, we compute:

$$\|\bar{B}_{\text{Broyden}} - B\|_F^2 = \left\| \frac{(y - Bs)s^\top}{s^\top s} \right\|_F^2 = \left\| (\bar{B} - B) \frac{ss^\top}{s^\top s} \right\|_F^2 \leq \|\bar{B} - B\|_F^2,$$

where we used the sub-multiplicativity of Frobenius norm and the fact that $\|ss^\top / (s^\top s)\|_F = 1$ in the last inequality. Thus, $\bar{B} = \bar{B}_{\text{Broyden}}$. $\square$

Broyden's update is characterized by two properties: it satisfies the secant condition and preserves the action on vectors orthogonal to s . Additionally, it is the unique minimal-change update in the Frobenius norm subject to the secant constraint.

SR1 Update

The Symmetric Rank-One (SR1) update [44] is a fundamental quasi-Newton method that maintains symmetry throughout the update process. The update formulas are given by

$$\bar{B}_{\text{SR1}} = B + \frac{(y - Bs)(y - Bs)^\top}{(y - Bs)^\top s}, \quad (2.29)$$

$$\bar{H}_{\text{SR1}} = H + \frac{(s - Hy)(s - Hy)^\top}{(s - Hy)^\top y}. \quad (2.30)$$

Derivation To derive eq. (2.29), we construct the updated matrix $\bar{B}$ as a rank-one update. That is, we assume that for some vector $z \in \mathbb{R}^n$,

$$\bar{B}_{\text{SR1}} = B + zz^\top. \quad (2.31)$$

For this update to satisfy the secant condition $\bar{B}_{\text{SR1}}s = y$, assuming $z^\top s \neq 0$, we require

$$Bs + zz^\top s = y, \quad (2.32)$$

which yields

$$z = \frac{y - Bs}{z^\top s}. \quad (2.33)$$

To determine $z^\top s$, we take the inner product of both sides of eq. (2.33) with s :

$$z^\top s = \frac{(y - Bs)^\top s}{z^\top s}. \quad (2.34)$$

Rearranging eq. (2.34) yields the key relation

$$(z^\top s)^2 = (y - Bs)^\top s. \quad (2.35)$$

Substituting eq. (2.35) into eq. (2.33) and using eq. (2.31), we obtain

$$\bar{B}_{\text{SR1}} = B + zz^\top \quad (2.36)$$

$$= B + \frac{(y - Bs)(y - Bs)^\top}{(z^\top s)^2} \quad (2.37)$$

$$= B + \frac{(y - Bs)(y - Bs)^\top}{(y - Bs)^\top s}, \quad (2.38)$$

which is the SR1 update formula stated in eq. (2.29).

Remarks The SR1 update requires $(y - Bs)^\top s \neq 0$ to be well-defined. When $(y - Bs)^\top s = 0$, the denominator in eq. (2.29) vanishes, and the update is typically skipped in practice. This situation can occur when the secant condition cannot be satisfied by a symmetric rank-one update, indicating that more sophisticated update strategies are needed.

Powell Symmetric Broyden (PSB) Update

The Powell Symmetric Broyden (PSB) update [27] is one of the most important quasi-Newton update rules. The update formulas are given by

$$\bar{B}_{\text{PSB}} = B + \frac{(y - Bs)s^\top + s(y - Bs)^\top}{s^\top s} - \frac{s^\top (y - Bs)}{(s^\top s)^2} ss^\top, \quad (2.39)$$

$$\bar{H}_{\text{PSB}} = H + \frac{(s - Hy)y^\top + y(s - Hy)^\top}{y^\top y} - \frac{y^\top (s - Hy)}{(y^\top y)^2} yy^\top. \quad (2.40)$$

Derivation To understand the motivation behind this formula, we note that in the SR1 update, the rank-one update was formulated symmetrically as $\bar{B}_{\text{SR1}} = B + zz^\top$. We may relax the requirement of maintaining symmetry throughout the update process. Instead, we can consider an asymmetric rank-one update of the form $zc^\top$, where the final result is symmetrized afterward.

For a given vector $c \in \mathbb{R}^n$ with $c^\top s \neq 0$, we define

$$z = \frac{y - Bs}{c^\top s} \quad (2.41)$$

and perform the asymmetric update

$$C_1 := B + \frac{(y - Bs)c^\top}{c^\top s}. \quad (2.42)$$

Since C_1 is generally not symmetric, we symmetrize it via

$$C_2 = \frac{C_1 + C_1^\top}{2}. \quad (2.43)$$

However, the symmetrized matrix C_2 may not satisfy the secant condition $C_2 s = y$. Therefore, we iterate this process:

$$\begin{cases} C_0 = B \\ C_{2t+1} = C_{2t} + \frac{(y - C_{2t}s)c^\top}{c^\top s} & \text{(asymmetric update)} \\ C_{2t+2} = \frac{C_{2t+1} + C_{2t+1}^\top}{2} & \text{(symmetrization)} \end{cases} \quad (2.44)$$

The key result is that the sequence $\{C_{2t}\}_{t=0}^\infty$ converges to a symmetric matrix satisfying the secant condition, as formalized in the following proposition.

Proposition 7 ([14], Lemma 7.2). *The sequence $\{C_{2t}\}_{t=0}^\infty$ defined by eq. (2.44) converges, and the limit is given by*

$$\lim_{t \rightarrow \infty} C_{2t} = B + \frac{(y - Bs)c^\top + c(y - Bs)^\top}{c^\top s} - \frac{(y - Bs)^\top s}{(c^\top s)^2} cc^\top. \quad (2.45)$$

Proof. We first analyze the even subsequence. Define

$$G_k := C_{2k}$$

for $k = 0, 1, 2, \dots$. By construction, each G_k is symmetric. From eq. (2.44) and the definition, we have

$$G_{k+1} = G_k + \frac{1}{2c^\top s} \left((y - G_k s)c^\top + c(y - G_k s)^\top \right).$$

Let us introduce the error vector

$$w_k := y - G_k s \quad (2.46)$$

and then we have

$$G_{k+1} = G_k + \frac{1}{2c^\top s} (w_k c^\top + c w_k^\top). \quad (2.47)$$

Substituting eq. (2.47) into eq. (2.46) yields

$$w_{k+1} = y - \left(G_k + \frac{1}{2c^\top s} (w_k c^\top + c w_k^\top) \right) s \quad (2.48)$$

$$= w_k - \frac{1}{2} w_k - \frac{w_k^\top s}{2c^\top s} c \quad (2.49)$$

$$= \frac{1}{2} \left(w_k - \frac{w_k^\top s}{c^\top s} c \right). \quad (2.50)$$

Hence

$$w_{k+1} = P w_k, \quad P := \frac{1}{2} \left(I - \frac{cs^\top}{c^\top s} \right). \quad (2.51)$$

The matrix $cs^\top/c^\top s$ has rank one and eigenvalues $1, 0, \dots, 0$. Therefore, P has one eigenvalue 0 and all remaining eigenvalues equal to $1/2$. In particular, its spectral radius is $1/2 < 1$. Thus, the Neumann series converges and

$$\sum_{k=0}^{\infty} w_k = \sum_{k=0}^{\infty} P^k (y - Bs) \quad (\text{from } w_0 = y - Bs) \quad (2.52)$$

$$= (I - P)^{-1} (y - Bs) \quad (2.53)$$

$$= 2 \left(I - \frac{1}{2} \frac{cs^\top}{c^\top s} \right) (y - Bs). \quad (\text{from the definition of } P) \quad (2.54)$$

Note that the last equation follows from

$$2(I - P) \left(I - \frac{1}{2} \frac{cs^\top}{c^\top s} \right) = \left(I + \frac{cs^\top}{c^\top s} \right) \left(I - \frac{1}{2} \frac{cs^\top}{c^\top s} \right) = I.$$

In particular, $\|w_k\| \rightarrow 0$ as $k \rightarrow \infty$. Hence, from eq. (2.47), we have

$$\lim_{k \rightarrow \infty} G_k = B + \frac{1}{2c^\top s} \sum_{k=0}^{\infty} (w_k c^\top + c w_k^\top) \quad (2.55)$$

$$= B + \left(\sum_{k=0}^{\infty} w_k \right) \frac{c^\top}{2c^\top s} + \frac{c}{2c^\top s} \left(\sum_{k=0}^{\infty} w_k \right)^\top \quad (2.56)$$

$$= B + \frac{(y - Bs)c^\top + c(y - Bs)^\top}{c^\top s} - \frac{(y - Bs)^\top s}{(c^\top s)^2} cc^\top. \quad (2.57)$$

Thus $G_k \rightarrow \bar{B}$. Finally,

$$C_{2k+1} = G_k + \frac{w_k c^\top}{c^\top s}. \quad (2.58)$$

Since $G_k \rightarrow \bar{B}$ and $\|w_k\| \rightarrow 0$, we obtain

$$C_{2k+1} - G_k \rightarrow 0. \quad (2.59)$$

Therefore both subsequences $\{C_{2k}\}$ and $\{C_{2k+1}\}$ converge to $\bar{B}$, and hence

$$C_k \rightarrow \bar{B}. \quad (2.60)$$

This completes the proof. $\square$

When $c = s$, the general formula in eq. (2.45) simplifies to the standard PSB update formula:

$$\bar{B}_{\text{PSB}} = B + \frac{(y - Bs)s^\top + s(y - Bs)^\top}{s^\top s} - \frac{(y - Bs)^\top s}{(s^\top s)^2} ss^\top, \quad (2.61)$$

which matches eq. (2.39).

Remarks The choice of $c = s$ is motivated by ensuring positive definiteness of the resulting update. The PSB update can be viewed as the limit of an iterative symmetrization process applied to asymmetric rank-one updates.

DFP Update

The Davidon–Fletcher–Powell (DFP) update [44] is a classical quasi-Newton update formula. The update formulas are given by

$$\bar{B}_{\text{DFP}} = \left(I - \frac{ys^\top}{y^\top s} \right) B \left(I - \frac{sy^\top}{y^\top s} \right) + \frac{yy^\top}{y^\top s}, \quad (2.62)$$

$$\bar{H}_{\text{DFP}} = H - \frac{Hy y^\top H}{y^\top H y} + \frac{ss^\top}{y^\top s}. \quad (2.63)$$

Derivation In the PSB update rule discussed earlier, we substituted $c = s$, but we can consider taking a different c . Specifically, we consider choosing c such that B_{k+1} becomes positive definite. Substituting $c = y$ yields the alternative form:

$$\bar{B}_{\text{DFP}} = B + \frac{(y - Bs)y^\top + y(y - Bs)^\top}{y^\top s} - \frac{(y - Bs)^\top s}{(y^\top s)^2} yy^\top. \quad (2.64)$$

BFGS Update

The Broyden–Fletcher–Goldfarb–Shanno (BFGS) update is one of the most widely used quasi-Newton methods. The update formulas are given by

$$\bar{B}_{\text{BFGS}} = B - \frac{Bss^\top B}{s^\top Bs} + \frac{yy^\top}{y^\top s}, \quad (2.65)$$

$$\bar{H}_{\text{BFGS}} = \left(I - \frac{sy^\top}{y^\top s} \right) H \left(I - \frac{ys^\top}{y^\top s} \right) + \frac{ss^\top}{y^\top s}. \quad (2.66)$$

Derivation This update can be derived by considering the dual of the DFP update. Specifically, we seek an update that minimizes the change in the inverse Hessian approximation H while satisfying the secant condition. Further details are provided in the following subsection.

2.3.3 BFGS Method

In this subsection, we focus on the BFGS update and provide a detailed derivation of its formula. BFGS update is known as one of the most successful quasi-Newton update rules in practice. Recall that the BFGS update formula is given by

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}. \quad (2.67)$$

Formula for H_k

The inverse of the BFGS update $H_k := B_k^{-1}$ is given by

$$H_{k+1} = \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) H_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}. \quad (2.68)$$

We now prove that eq. (2.68) indeed gives the inverse of the BFGS update.

Proposition 8. Equation (2.68) gives the exact inverse of the BFGS update, i.e., $H_{k+1} = B_{k+1}^{-1}$.

Proof. The BFGS update can be rewritten in a compact rank-two form

$$B_{k+1} = B_k + UCV^\top \quad (2.69)$$

where

$$U = [B_k s_k \quad y_k], \quad C = \begin{pmatrix} -\frac{1}{s_k^\top B_k s_k} & 0 \\ 0 & \frac{1}{y_k^\top s_k} \end{pmatrix}, \quad V = [B_k s_k \quad y_k]$$

since

$$UCV^\top = \begin{bmatrix} -\frac{B_k s_k}{s_k^\top B_k s_k} & \frac{y_k}{y_k^\top s_k} \end{bmatrix} \begin{bmatrix} s_k^\top B_k \\ y_k^\top \end{bmatrix} = -\frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}. \quad (2.70)$$

By the Sherman–Morrison–Woodbury identity, we have

$$\begin{aligned}
H_{k+1} &= (B_k + UCV^\top)^{-1} \\
&= B_k^{-1} - B_k^{-1}U \left(C^{-1} + V^\top B_k^{-1}U \right)^{-1} V^\top B_k^{-1} \\
&= H_k - [s_k \quad H_k y_k] \left(\begin{pmatrix} -s_k^\top B_k s_k & 0 \\ 0 & y_k^\top s_k \end{pmatrix} + \begin{pmatrix} s_k^\top B_k s_k & s_k^\top y_k \\ y_k^\top s_k & y_k^\top H_k y_k \end{pmatrix} \right)^{-1} \begin{bmatrix} s_k^\top \\ y_k^\top H_k \end{bmatrix} \\
&= H_k - [s_k \quad H_k y_k] \begin{pmatrix} 0 & y_k^\top s_k \\ y_k^\top s_k & y_k^\top H_k y_k + y_k^\top s_k \end{pmatrix}^{-1} \begin{bmatrix} s_k^\top \\ y_k^\top H_k \end{bmatrix} \\
&= H_k - [s_k \quad H_k y_k] \left(-\frac{1}{(y_k^\top s_k)^2} \begin{pmatrix} y_k^\top H_k y_k + y_k^\top s_k & -y_k^\top s_k \\ -y_k^\top s_k & 0 \end{pmatrix} \right) \begin{bmatrix} s_k^\top \\ y_k^\top H_k \end{bmatrix} \\
&= H_k + \frac{1}{(y_k^\top s_k)^2} [s_k \quad H_k y_k] \begin{pmatrix} y_k^\top H_k y_k + y_k^\top s_k & -y_k^\top s_k \\ -y_k^\top s_k & 0 \end{pmatrix} \begin{bmatrix} s_k^\top \\ y_k^\top H_k \end{bmatrix} \\
&= H_k + \frac{1}{(y_k^\top s_k)^2} \left((y_k^\top H_k y_k + y_k^\top s_k) s_k s_k^\top - (y_k^\top s_k)(s_k y_k^\top H_k + H_k y_k s_k^\top) \right) \\
&= \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) H_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) + \frac{s_k s_k^\top}{y_k^\top s_k}.
\end{aligned}$$

which completes the proof. $\square$

Positive Definiteness of B_{k+1}

An important property of the BFGS update is that if the current approximation B_k is positive definite and the curvature condition $y_k^\top s_k > 0$ holds, then the updated approximation B_{k+1} is also guaranteed to be positive definite.

Proposition 9. *If B_k is positive definite and $y_k^\top s_k > 0$, then B_{k+1} defined by eq. (2.67) is also positive definite.*

Proof. By assumption, B_k and its inverse H_k are positive definite. For any nonzero vector $v \in \mathbb{R}^n$, we have

$$\begin{aligned}
v^\top H_{k+1} v &= v^\top \left(I - \frac{s_k y_k^\top}{y_k^\top s_k} \right) H_k \left(I - \frac{y_k s_k^\top}{y_k^\top s_k} \right) v + v^\top \frac{s_k s_k^\top}{y_k^\top s_k} v \\
&\geq 0 + \frac{(s_k^\top v)^2}{y_k^\top s_k} > 0,
\end{aligned}$$

where the first term is nonnegative since H_k is positive definite, and the second term is positive due to the curvature condition $y_k^\top s_k > 0$. Thus, H_{k+1} is positive definite, and consequently, $B_{k+1} = H_{k+1}^{-1}$ is also positive definite. $\square$

KL

We next present a variational characterization of the BFGS update via the Kullback–Leibler (KL) divergence. For zero-mean multivariate Gaussians $\mathcal{N}(0, A^{-1})$ and $\mathcal{N}(0, B^{-1})$, the expression

$$\psi(A) = \text{tr}(A) - \log \det(A)$$

coincides with the KL divergence up to the additive constant $-n$, so minimizing ψ is equivalent to minimizing the KL distance. Then, the BFGS update is given by the solution to the optimization

problem

$$\begin{aligned} & \underset{B \in \text{PD}(n)}{\text{minimize}} && \psi(B_k^{-1/2} B B_k^{-1/2}) \\ & \text{subject to} && B s_k = y_k \end{aligned}$$

The constraint $B s_k = y_k$ in the optimization problem is precisely the secant condition discussed earlier, ensuring that the updated matrix interpolates the latest curvature pair.

See literature for proof of these formulations [31, 32]

Trace and Determinant Formulas for the BFGS Update

We also present trace and determinant formulas for the BFGS update, which are useful in analyzing the eigenvalue behavior of the updated matrix. For Hermitian matrices, these quantities correspond to the sum and product of eigenvalues, respectively. Hence, if both the trace and the determinant are appropriately bounded, one may expect the eigenvalues themselves to remain bounded (provided all eigenvalues are positive, for instance). This is closely related to assumptions on the Hessian eigenvalues of the objective function, such as μ -strong convexity and L -smoothness.

Trace Formula The trace of the BFGS-updated matrix has an explicit formula.

Proposition 10 ([44, (6.44)]). *Let $B_+ = B - \frac{B s s^\top B}{s^\top B s} + \frac{y y^\top}{y^\top s}$ be the BFGS update. Then*

$$\text{tr}(B_+) = \text{tr}(B) - \frac{\|B s\|^2}{s^\top B s} + \frac{\|y\|^2}{y^\top s}. \quad (2.71)$$

Proof. Applying the trace to the BFGS update formula, we have

$$\text{tr}(B_+) = \text{tr}(B) - \text{tr}\left(\frac{B s s^\top B}{s^\top B s}\right) + \text{tr}\left(\frac{y y^\top}{y^\top s}\right). \quad (2.72)$$

For the second term in eq. (2.72), note that

$$\text{tr}\left(\frac{B s s^\top B}{s^\top B s}\right) = \frac{1}{s^\top B s} \text{tr}\left((B s)(B s)^\top\right) = \frac{\|B s\|^2}{s^\top B s}. \quad (2.73)$$

For the third term in eq. (2.72), we have

$$\text{tr}\left(\frac{y y^\top}{y^\top s}\right) = \frac{1}{y^\top s} \text{tr}(y y^\top) = \frac{\|y\|^2}{y^\top s}. \quad (2.74)$$

Substituting eqs. (2.73) and (2.74) into eq. (2.72) yields eq. (2.71). $\square$

Determinant Formula The determinant of the BFGS-updated matrix also admits a closed-form expression.

Proposition 11 ([44, (6.45)]). *Let $B_+ = B - \frac{B s s^\top B}{s^\top B s} + \frac{y y^\top}{y^\top s}$ be the BFGS update and assume that B is nonsingular. Then*

$$\det(B_+) = \det(B) \frac{y^\top s}{s^\top B s}. \quad (2.75)$$

Proof. Recall the rank-two formulation of the BFGS update defined in eq. (2.69). By the matrix determinant lemma, the matrices U, C, V in eq. (2.69) satisfy

$$\det(B_{k+1}) = \det\left(B_k + U C V^\top\right) = \det(B_k) \det(C) \det\left(C^{-1} + V^\top B_k^{-1} U\right), \quad (2.76)$$

where I_2 is the 2×2 identity matrix. Since $U = V = [B_k s_k \ y_k]$, we have

$$V^\top B_k^{-1} U = \begin{bmatrix} s_k^\top B_k s_k & s_k^\top y_k \\ y_k^\top s_k & y_k^\top B_k^{-1} y_k \end{bmatrix}$$

and thus

$$C^{-1} + V^\top B_k^{-1} U = \begin{bmatrix} -s_k^\top B_k s_k & 0 \\ 0 & y_k^\top s_k \end{bmatrix} + \begin{bmatrix} s_k^\top B_k s_k & s_k^\top y_k \\ y_k^\top s_k & y_k^\top B_k^{-1} y_k \end{bmatrix} = \begin{bmatrix} 0 & s_k^\top y_k \\ y_k^\top s_k & y_k^\top B_k^{-1} y_k \end{bmatrix}. \quad (2.77)$$

Thus, combining eqs. (2.76) and (2.77), we have

$$\det(B_{k+1}) = \det(B_k) \left(-\frac{1}{s_k^\top B_k s_k} \cdot \frac{1}{y_k^\top s_k} \right) \left(-(s_k^\top y_k)(y_k^\top s_k) \right) = \det(B_k) \frac{y_k^\top s_k}{s_k^\top B_k s_k},$$

which completes the proof. $\square$

2.3.4 BFGS vs DFP

Although BFGS and DFP are quite symmetric in structure, they exhibit remarkably different practical efficiency when applied to real optimization problems. Powell's analysis [47] investigates this asymmetry by studying the behavior of both methods on a simple two-dimensional quadratic function. While asymptotic convergence theory often suggests that both methods should perform similarly, Powell reveals that their practical efficiency can differ dramatically, especially when the approximate Hessian is far from the true Hessian.

Problem Setup

Following Powell's framework, we consider the quadratic function

$$f(x, y) = \frac{1}{2}(x^2 + y^2), \quad (2.78)$$

with both BFGS and DFP methods using a fixed step size of $\alpha_k = 1$ at every iteration. This choice of unit step size is practical for quadratic functions, where it often satisfies standard line search criteria.

The initial Hessian approximation B_0 is chosen to have eigenvalues 1 and λ_1 , where λ_1 represents the degree of error in the initial approximation. The initial point x_0 is selected as

$$\theta = \arctan(\sqrt{\lambda_1}), \quad x_0 = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}, \quad (2.79)$$

which aligns with Powell's original analysis. See [47] for details on this choice.

The iteration continues until the norm of the current point falls below a tolerance relative to its initial norm. For each value of λ_1 , we record the number of iterations required for convergence.

Experimental Results

The numerical results are presented in Table 2.1, which partially reproduces the content of Powell's original tables. The convergence behavior depends critically on the initial eigenvalue λ_1 :

Figure 2.8 and Figure 2.9 illustrate the iterative trajectories for specific values of λ_1 . These figures visualize how the two methods navigate toward the minimum (at the origin) from the same initial point, clearly showing the difference in convergence speed and path.

λ_1	BFGS	DFP
0.001	4	3
0.01	5	3
0.1	6	4
1	1	1
10	8	16
100	10	107
1000	12	1006
10000	15	9987

Table 2.1: Convergence comparison between BFGS and DFP methods for different initial eigenvalues λ_1

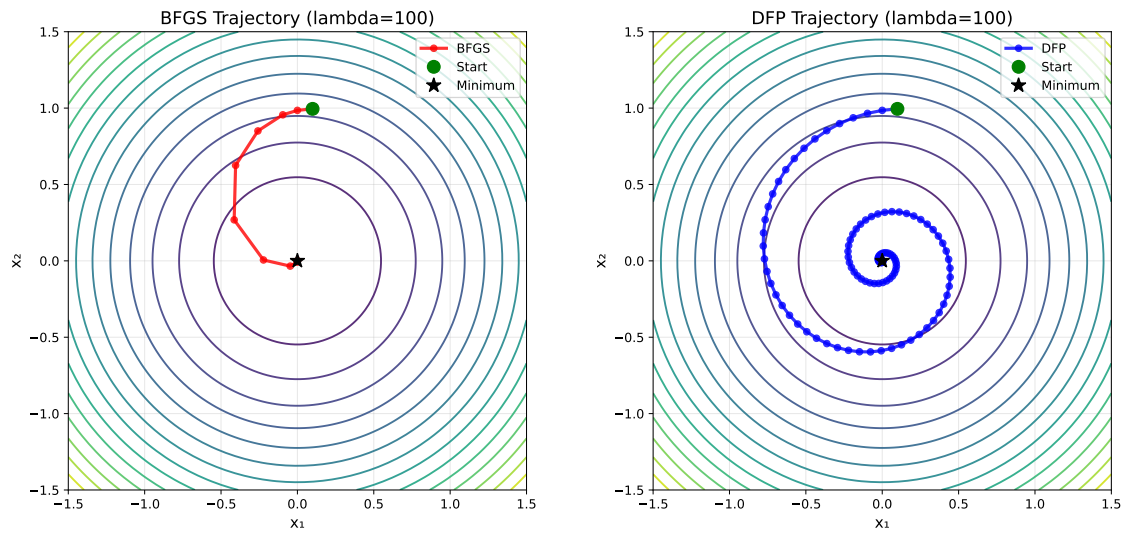


Fig. 2.8: Iterative trajectories for BFGS and DFP methods when $\lambda_1 = 100$. BFGS converges in 10 iterations while DFP requires 107 iterations, showing BFGS's superior efficiency for large eigenvalue errors.

Analysis and Discussion

The numerical results reveal a striking asymmetry between BFGS and DFP. When $\lambda_1 > 1$ (i.e., the initial Hessian approximation overestimates the true Hessian curvature), BFGS demonstrates significantly better efficiency. Conversely, when $\lambda_1 < 1$ (i.e., the initial Hessian approximation underestimates the true curvature), DFP performs slightly better than BFGS, though the difference is modest. This trend reversal is predicted by the theoretical symmetry between the two methods, but the magnitude of the difference is notable.

Asymmetry in Hessian Correction The asymmetry of the performance arises from a fundamental difference in how these two methods correct erroneous eigenvalues. The core insight is that correcting erroneously large eigenvalues is more critical than correcting erroneously small ones.

When a Hessian eigenvalue is overestimated, the algorithm takes steps that are too conservative, resulting in slow progress toward the minimum. Correcting such errors requires the update formula to reduce these large eigenvalues toward one. The BFGS update is highly effective at this task.

In contrast, when a Hessian eigenvalue is underestimated, the algorithm takes steps that are

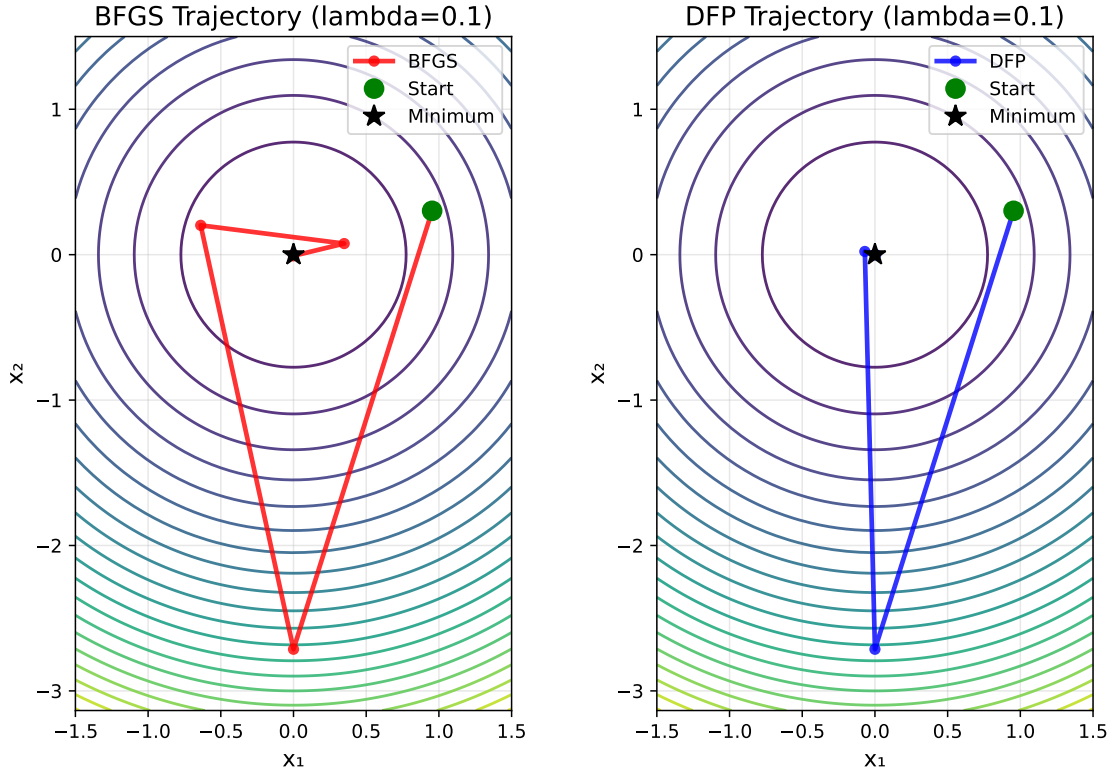


Fig. 2.9: Iterative trajectories for BFGS and DFP methods when $\lambda_1 = 0.1$. Both methods converge quickly, with DFP slightly faster than BFGS, demonstrating the symmetric behavior when eigenvalues are underestimated.

slightly too aggressive, but the error is self-correcting. Subsequent gradient computations provide information that helps refine the approximation. Thus, correcting underestimated eigenvalues is inherently easier and requires fewer iterations.

The DFP update struggles with large eigenvalues. In the worst case, it can reduce a large eigenvalue by only a small amount per iteration, potentially requiring as many iterations as the magnitude of the eigenvalue itself. This explains why DFP's performance degrades catastrophically as λ_1 increases beyond one.

Practical Implications These findings provide strong empirical justification for the widespread practical preference for BFGS over DFP. The analysis of Powell's simple quadratic problem yields deep insights about quasi-Newton methods' behavior far from the solution, where most computational effort is expended. The superior efficiency of BFGS in correcting erroneous Hessian approximations makes it the algorithm of choice for robust and efficient unconstrained optimization.

2.3.5 Limited Memory BFGS (L-BFGS)

Limited-memory quasi-Newton methods extend classical quasi-Newton methods to large-scale optimization problems. In standard quasi-Newton methods, the approximate Hessian or inverse Hessian matrix is stored and updated as a dense matrix, requiring $\mathcal{O}(n^2)$ memory for n variables.

The L-BFGS method [37], which is based on the BFGS update, avoids storing the full matrix explicitly. Instead, it maintains only the most recent m vector pairs $\{(s_i, y_i)\}$. This reduces the storage requirement to $\mathcal{O}(nm)$, which is a dramatic improvement when m is a small constant

Algorithm 1: L-BFGS Two-Loop Recursion for $r = H_m q$ [44, Algorithm 7.4]

Input: q , stored pairs $\{(s_i, y_i)\}_{i=0}^{m-1}$, initial matrix H_0
Output: $r = H_m q$

```

1 for  $i = m-1, m-2, \dots, 0$  do
2    $\rho_i \leftarrow 1/(y_i^\top s_i)$ 
3    $\alpha_i \leftarrow \rho_i s_i^\top q$ 
4    $q \leftarrow q - \alpha_i y_i$ 
5 end
6  $r \leftarrow H_0 q$ 
7 for  $i = 0, \dots, m-1$  do
8    $\beta_i \leftarrow \rho_i y_i^\top r$ 
9    $r \leftarrow r + s_i(\alpha_i - \beta_i)$ 
10 end
11 return  $r$ 

```

(typically $m \leq 10$).

Throughout this subsection, we work with a finite sequence of matrices

$$H_0, H_1, \dots, H_m,$$

where H_ℓ denotes the inverse Hessian approximation obtained after ℓ BFGS updates applied to a given initial matrix H_0 . Note that this is not the same as the iterates in an optimization algorithm. Here, we focus solely on the structure of the BFGS updates.

Compact Representation of H_m

Let $\{(s_i, y_i)\}_{i=0}^{m-1}$ be the stored correction pairs, and define

$$\rho_i = \frac{1}{y_i^\top s_i}, \quad V_i = I - \rho_i y_i s_i^\top. \quad (2.80)$$

From eq. (2.68), the BFGS update for the inverse Hessian can be expressed as

$$H_{i+1} = V_i^\top H_i V_i + \rho_i s_i s_i^\top, \quad (2.81)$$

for $i = 0, \dots, m-1$. By recursively expanding this relation, we obtain the compact representation

$$H_m = V_{m-1}^\top \cdots V_0^\top H_0 V_0 \cdots V_{m-1} + \sum_{j=0}^{m-1} (V_{m-1}^\top \cdots V_{j+1}^\top) \rho_j s_j s_j^\top (V_{j+1} \cdots V_{m-1}), \quad (2.82)$$

where H_0 denotes the chosen initial inverse Hessian approximation, typically a scaled identity matrix.

Two-Loop Recursion

The compact representation above can be applied to any vector q without explicitly forming H_m . Let $r = H_m q$. Exploiting the associative structure of the matrix products, this operation can be carried out using two short loops of length m , leading to the well-known L-BFGS two-loop recursion [44, Algorithm 7.4]. The pseudocode is provided in Algorithm 1. This algorithm requires $\mathcal{O}(md)$ arithmetic operations and $\mathcal{O}(md)$ storage, where d denotes the problem dimension.

Now, we verify that the output of Algorithm 1 indeed computes $r = H_m q$.

Proposition 12. *The output of the two-loop recursion Algorithm 1 satisfies $r = H_m q$.*

Proof. In the backward recursion, for $i = m-1, m-2, \dots, 0$, from the input vector $q^{(m)} := q$, the algorithm computes

$$\alpha_i = \rho_i s_i^\top q^{(i+1)}, \quad q^{(i)} := q^{(i+1)} - \alpha_i y_i. \quad (2.83)$$

Substituting the definition of α_i and using eq. (2.80), we find

$$q^{(i)} = q^{(i+1)} - \rho_i \left(s_i^\top q^{(i+1)} \right) y_i = \left(I - \rho_i y_i s_i^\top \right) q^{(i+1)} = V_i q^{(i+1)}.$$

Thus, for all $i = 0, 1, \dots, m-1$, we have

$$q^{(i)} = V_i V_{i+1} \cdots V_{m-1} q.$$

Then, the algorithm applies the initial inverse Hessian approximation:

$$r^{(0)} = H_0 q^{(0)} = H_0 V_0 V_1 \cdots V_{m-1} q.$$

Next, for $i = 0, 1, \dots, m-1$, the forward recursion computes

$$\beta_i = \rho_i y_i^\top r^{(i)}, \quad r^{(i+1)} = r^{(i)} + s_i (\alpha_i - \beta_i).$$

Substituting the definitions of α_i , β_i , and $q^{(i+1)}$, we have

$$\begin{aligned} r^{(i+1)} &= r^{(i)} + \rho_i s_i s_i^\top (V_{i+1} V_{i+2} \cdots V_{m-1}) q - \rho_i s_i y_i^\top r^{(i)} \\ &= \left(I - \rho_i y_i s_i^\top \right) r^{(i)} + \rho_i s_i s_i^\top (V_{i+1} V_{i+2} \cdots V_{m-1}) q \\ &= V_i^\top r^{(i)} + \rho_i s_i s_i^\top (V_{i+1} V_{i+2} \cdots V_{m-1}) q. \end{aligned}$$

By recursively expanding this relation from the initial value $r^{(0)} = H_0 q^{(0)}$, we obtain

$$r^{(m)} = V_{m-1}^\top \cdots V_0^\top H_0 V_0 \cdots V_{m-1} q + \sum_{j=0}^{m-1} (V_{m-1}^\top \cdots V_{j+1}^\top) \rho_j s_j s_j^\top (V_{j+1} \cdots V_{m-1}) q, \quad (2.84)$$

which matches the compact representation of H_m in eq. (2.82) applied to q , completing the proof. $\square$

Thus, the two-loop recursion exactly evaluates the action of H_m on a vector while avoiding explicit matrix construction, achieving both mathematical rigor and computational efficiency.

Initial Step Size

A crucial component of the L-BFGS method is the choice of the initial matrix H_0 . A widely used and well-justified option is a scaled identity,

$$H_0 = \gamma I, \quad (2.85)$$

where the scaling parameter is chosen as

$$\gamma = \frac{s_{m-1}^\top y_{m-1}}{y_{m-1}^\top y_{m-1}}. \quad (2.86)$$

This choice is motivated by the relationship between the approximate inverse Hessian and the local curvature of the objective function [37, 51]. To justify this scaling, assume that the objective

function f is twice continuously differentiable and consider the mean Hessian along the most recent step:

$$\bar{G} = \int_0^1 \nabla^2 f(x + \tau s_{m-1}) d\tau, \quad (2.87)$$

where s_{m-1} denotes the most recent displacement. By the mean value theorem,

$$y_{m-1} = \nabla f(x + s_{m-1}) - \nabla f(x) = \bar{G} s_{m-1}.$$

Using this relation, the scaling factor can be written as

$$\frac{s_{m-1}^\top y_{m-1}}{y_{m-1}^\top y_{m-1}} = \frac{(\bar{G}^{1/2} s_{m-1})^\top (\bar{G}^{1/2} s_{m-1})}{(\bar{G}^{1/2} s_{m-1})^\top \bar{G} (\bar{G}^{1/2} s_{m-1})}, \quad (2.88)$$

which is a Rayleigh quotient. If $\bar{G}^{1/2} s_{m-1}$ happens to be an eigenvector of $\bar{G}$, then the reciprocal of this quantity equals the corresponding eigenvalue.

Moreover, the choice (2.86) coincides with the short step size of the Barzilai–Borwein method [4], highlighting a close connection between L-BFGS initialization and classical step-length selection strategies. This observation further supports the effectiveness of the scaled-identity initialization in practice.

2.4 Modified Secant Condition

This section explains the modified secant condition, a modification of the standard secant condition used in quasi-Newton methods. The modified secant condition incorporates function value information in addition to gradient information, leading to more accurate Hessian approximations.

2.4.1 Standard Secant Condition

First, we review the standard secant condition used in quasi-Newton methods. Let x_k and x_{k+1} be two consecutive iterates generated by an optimization algorithm. To compute the next iterate, we need to update the approximate Hessian matrix B_k to B_{k+1} . Define the step and the gradient difference as

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

The standard approach is to update B_k to satisfy the secant condition:

$$B_{k+1} s_k = y_k. \quad (2.89)$$

To justify this condition, consider a quadratic approximation model around x_{k+1} :

$$m_{k+1}(x) = f(x_{k+1}) + \nabla f(x_{k+1})^\top (x - x_{k+1}) + \frac{1}{2} (x - x_{k+1})^\top B_{k+1} (x - x_{k+1}). \quad (2.90)$$

By construction, this model satisfies

$$\begin{cases} m_{k+1}(x_{k+1}) = f(x_{k+1}) \\ \nabla m_{k+1}(x_{k+1}) = \nabla f(x_{k+1}) \end{cases} \quad (2.91)$$

regardless of the choice of B_{k+1} . Additionally, the secant condition eq. (2.89) ensures that the model satisfies

$$\nabla m_{k+1}(x_k) = \nabla f(x_{k+1}) - B_{k+1} (x_{k+1} - x_k) \quad (\text{by eq. (2.90)}) \quad (2.92)$$

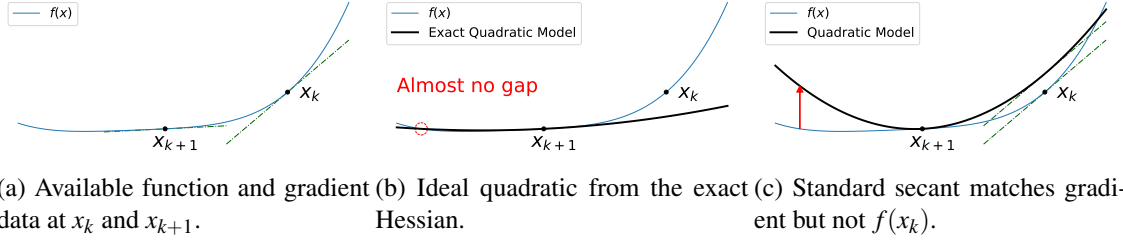


Fig. 2.10: Motivation for modified secant equations. Combining function values with gradients (a) aims to approximate the ideal Newton model (b), while the standard secant update (c) omits $f(x_k)$ and can misestimate curvature.

$$= \nabla f(x_{k+1}) - (\nabla f(x_{k+1}) - \nabla f(x_k)) \quad (\text{by secant condition (2.89)}) \quad (2.93)$$

$$= \nabla f(x_k) \quad (2.94)$$

which matches the gradient information from the previous iteration. Thus, by using the secant condition, we ensure that the quadratic model $m_{k+1}(x)$ accurately reflects the gradient information at both x_{k+1} and x_k , except for the previous function value $f(x_k)$.

2.4.2 Modified Secant Condition

Although the secant condition is widely used, it can fail to capture the curvature of the objective function accurately because it only matches gradient information. We illustrate this limitation in Figure 2.10. In Figure 2.10a, we have two points x_k and x_{k+1} with their corresponding function values and gradients. In Figure 2.10b, we see that the ideal quadratic model constructed from the exact Hessian fits well around the new point x_{k+1} , leading to a better convergence behavior. However, in Figure 2.10c, the standard secant update only matches the gradients at these points, neglecting the function value at x_k . It can severely misestimate the curvature, leading to a poor approximation of the true objective function.

We can overcome this limitation by utilizing the function values. The basic idea is illustrated in Figure 2.11. Even when the gradients are the same at two points x_k and x_{k+1} , natural interpolation of function values differs depending on the function values $f(x_k)$ and $f(x_{k+1})$. This observation motivates the modified secant condition, which incorporates function value information to improve Hessian approximation.

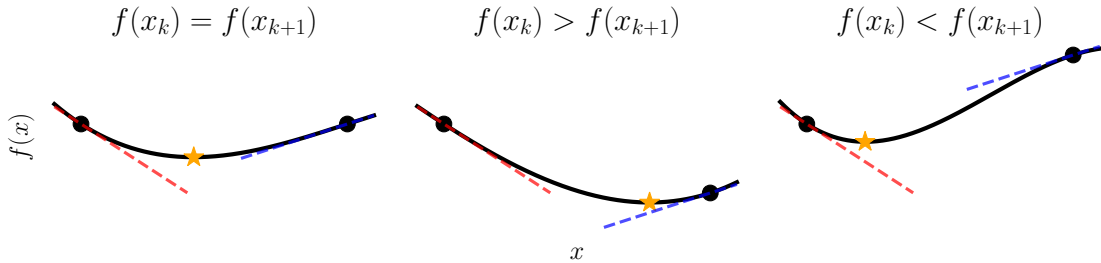


Fig. 2.11: Interpolation with identical gradients at x_k and x_{k+1} but different function values. This leads to distinct interpolants, highlighting the importance of incorporating function value information in Hessian approximation.

In the following, we present two well-known modified secant conditions.

Function-Value-Based Modified Secant Condition

The first modification incorporates the function value at the previous point into the quadratic model [1, 59, 63]. Let us consider an alternate model with a different approximate Hessian B_{k+1}^F :

$$m_{k+1}^F(x) = f(x_{k+1}) + \nabla f(x_{k+1})^\top (x - x_{k+1}) + \frac{1}{2}(x - x_{k+1})^\top B_{k+1}^F (x - x_{k+1}). \quad (2.95)$$

In addition to eq. (2.91), we require this model to satisfy

$$m_{k+1}^F(x_k) = f(x_k), \quad (2.96)$$

which enforces that the function value at the previous point is correctly modeled. This is the key idea behind function-value-based modified secant conditions.

Let us derive the corresponding modified secant condition for B_{k+1}^F . Substituting the definition of m_{k+1}^F from eq. (2.95) and using $s_k = x_{k+1} - x_k$ into eq. (2.96), the condition becomes

$$\begin{aligned} f(x_k) &= f(x_{k+1}) + \nabla f(x_{k+1})^\top (x_k - x_{k+1}) + \frac{1}{2}(x_k - x_{k+1})^\top B_{k+1}^F (x_k - x_{k+1}) \\ &= f(x_{k+1}) - \nabla f(x_{k+1})^\top s_k + \frac{1}{2}s_k^\top B_{k+1}^F s_k. \end{aligned} \quad (2.97)$$

Now, assume that the updated matrix has the form

$$B_{k+1}^F s_k = y_k - \sigma_k^F s_k, \quad (2.98)$$

where $\sigma_k^F \in \mathbb{R}$ is a scalar to be determined. Substituting eq. (2.98) into eq. (2.97) and using $s_k^\top B_{k+1}^F s_k = s_k^\top y_k - \sigma_k^F \|s_k\|^2$ gives

$$f(x_k) = f(x_{k+1}) - \nabla f(x_{k+1})^\top s_k + \frac{1}{2}s_k^\top y_k - \frac{\sigma_k^F}{2}\|s_k\|^2. \quad (2.99)$$

Solving for σ_k^F and using $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, we obtain

$$\sigma_k^F = \frac{2(f(x_{k+1}) - f(x_k)) - (2\nabla f(x_{k+1}) - y_k)^\top s_k}{\|s_k\|^2} \quad (2.100)$$

$$= \frac{2(f(x_{k+1}) - f(x_k)) - (\nabla f(x_{k+1}) + \nabla f(x_k))^\top s_k}{\|s_k\|^2}. \quad (2.101)$$

Therefore, the modified secant condition eq. (2.98) becomes

$$B_{k+1}^F s_k = y_k + \frac{2(f(x_k) - f(x_{k+1})) + (\nabla f(x_{k+1}) + \nabla f(x_k))^\top s_k}{\|s_k\|^2} s_k. \quad (2.102)$$

To preserve the possibility of maintaining positive definiteness under a BFGS-type update when $s_k^\top y_k > 0$, we can modify the formula by taking the maximum with zero in the numerator.

$$B_{k+1}^{F'} s_k = y_k + \frac{\max(0, 2(f(x_k) - f(x_{k+1})) + (\nabla f(x_{k+1}) + \nabla f(x_k))^\top s_k)}{\|s_k\|^2} s_k. \quad (2.103)$$

This is the function-value-based modified secant condition. Unlike the standard secant condition, this formulation ensures that the quadratic model matches not only the function value at the previous point but also the gradients at both points. Refer to Figure 2.12 for an illustration.

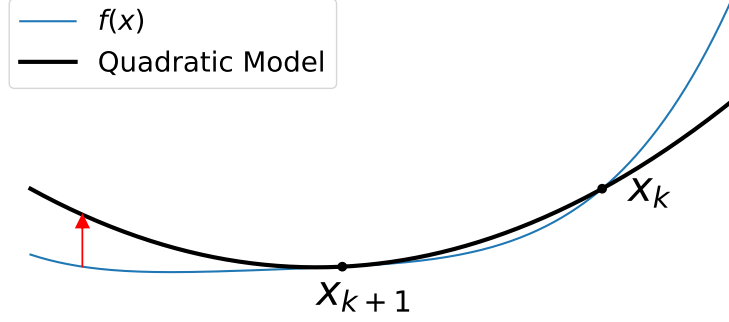


Fig. 2.12: Function-value-based modified secant equation. The modified secant equation $B_k^F s_k = y_k - \sigma_k^F s_k$ constructs a quadratic model that satisfies the function value condition $m_k^F(x_{k-1}) = f(x_{k-1})$ at the previous point. This differs from the standard secant equation, which only matches gradients but not function values.

Cubic-Augmented Modified Secant Condition

The second modification introduces a cubic term to the model, allowing simultaneous satisfaction of both function value and gradient matching at the previous point [61, 65, 66]. Let $T_{k+1} \in \mathbb{R}^{n \times n \times n}$ be the third-order derivative tensor of f at x_{k+1} such that

$$s_k^\top (T_{k+1} s_k) s_k = \sum_{i,j,l=1}^n \partial_{x_i x_j x_l} f(x_{k+1}) s_k^{(i)} s_k^{(j)} s_k^{(l)}, \quad (2.104)$$

where $\partial_{x_i x_j x_l} f$ denotes the third derivative of f with respect to x_i , x_j , and x_l , and $s_k^{(i)}$ is the i -th component of the vector s_k . We introduced this tensor solely for analysis purposes, and it will be eliminated from the final formula.

By incorporating this tensor term, we can define the following cubic-augmented model:

$$\begin{aligned} m_{k+1}^C(x) = & f(x_{k+1}) + \nabla f(x_{k+1})^\top (x - x_{k+1}) + \frac{1}{2} (x - x_{k+1})^\top B_{k+1}^C (x - x_{k+1}) \\ & + \frac{1}{6} (x - x_{k+1})^\top (T_{k+1} (x - x_{k+1})) (x - x_{k+1}). \end{aligned} \quad (2.105)$$

With this model, we can enforce both function value and gradient matching at the previous point x_k . Specifically, we require

$$\begin{cases} m_{k+1}^C(x_k) = f(x_k) \\ \nabla m_{k+1}^C(x_k) = \nabla f(x_k) \end{cases} \quad (2.106)$$

By substituting the definition of m_{k+1}^C from eq. (2.105) and using $s_k = x_{k+1} - x_k$, we can rewrite these conditions as

$$f(x_k) = f(x_{k+1}) - s_k^\top \nabla f(x_{k+1}) + \frac{1}{2} s_k^\top B_{k+1}^C s_k - \frac{1}{6} s_k^\top (T_{k+1} s_k) s_k, \quad (2.107)$$

$$\nabla f(x_k) = \nabla f(x_{k+1}) - B_{k+1}^C s_k + \frac{1}{2} (T_{k+1} s_k) s_k. \quad (2.108)$$

Rearranging eq. (2.107) and projecting eq. (2.108) onto s_k gives

$$3(f(x_k) - f(x_{k+1})) + 3s_k^\top \nabla f(x_{k+1}) = \frac{3}{2} s_k^\top B_{k+1}^C s_k - \frac{1}{2} s_k^\top (T_{k+1} s_k) s_k, \quad (2.109)$$

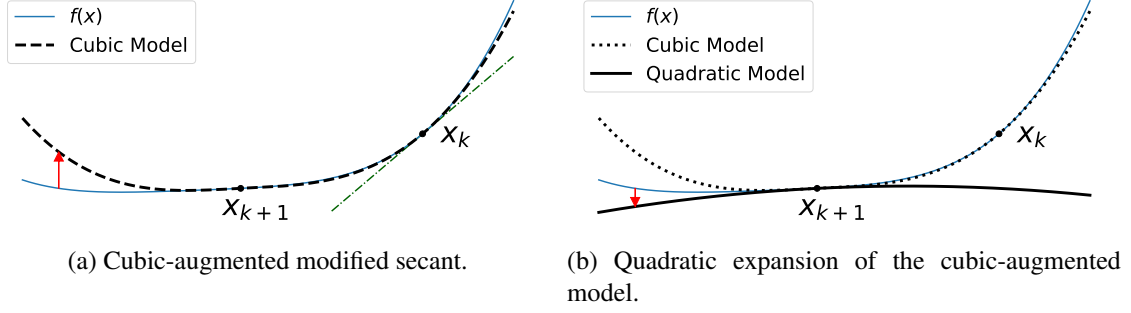


Fig. 2.13: Cubic-augmented modified secant equation. In Figure 2.13a, by incorporating a cubic term $\eta \|x - x_k\|^3$ in the model, the modified secant equation $B_k^C s_k = y_k + \sigma_k^C s_k$ enables simultaneous satisfaction of both conditions at the previous point. In Figure 2.13b, while the cubic model satisfies both function value and gradient conditions, its underlying quadratic component may be indefinite or negative definite.

$$-s_k^\top y_k = -s_k^\top B_{k+1}^C s_k + \frac{1}{2} s_k^\top (T_{k+1} s_k) s_k. \quad (2.110)$$

By summing up eqs. (2.109) and (2.110) and using $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, we eliminate the tensor term and obtain the scalar identity

$$3(f(x_k) - f(x_{k+1})) + \frac{3}{2} s_k^\top (\nabla f(x_{k+1}) + \nabla f(x_k)) + \frac{1}{2} s_k^\top y_k = \frac{1}{2} s_k^\top B_{k+1}^C s_k. \quad (2.111)$$

Assume again that $B_{k+1}^C s_k$ is a linear combination of y_k and s_k , i.e., $B_{k+1}^C s_k = y_k + \sigma_k^C s_k$ for some scalar σ_k^C . Then, eq. (2.111) yields

$$\sigma_k^C = \frac{6(f(x_k) - f(x_{k+1})) + 3s_k^\top (\nabla f(x_k) + \nabla f(x_{k+1}))}{\|s_k\|^2}. \quad (2.112)$$

Plugging eq. (2.112) back into the ansatz yields the cubic-augmented modified secant condition:

$$B_{k+1}^C s_k = y_k + \frac{6(f(x_k) - f(x_{k+1})) + 3s_k^\top (\nabla f(x_k) + \nabla f(x_{k+1}))}{\|s_k\|^2} s_k. \quad (2.113)$$

This is a cubic-augmented modified secant condition. Unlike the function-value-based quadratic modification, this formulation allows simultaneous satisfaction of both the function value and gradient conditions at the previous point through the introduction of the tensor term. Refer to Figure 2.13 for an illustration.

2.4.3 Other Curvature Storage Methods

For storing the curvature information, there are several topics. Agg-BFGS [8] is an alternative approach to managing curvature information by aggregating data rather than discarding the oldest information and adding the newest. Multi-Secant [34] extends the secant condition framework by maintaining multiple pairs of step and gradient difference vectors. In the standard formulation, we define

$$s_i = x_{i+1} - x_i, \quad y_i = \nabla f(x_{i+1}) - \nabla f(x_i). \quad (i = k-m, \dots, k)$$

An alternative anchored formulation centers all vectors at the most recent iterate:

$$s_i = x_{k+1} - x_i, \quad y_i = \nabla f(x_{k+1}) - \nabla f(x_i). \quad (i = k-m, \dots, k)$$

This anchored approach provides a different perspective on utilizing historical information for improved Hessian approximation.

Chapter 3

Noise-Tolerant Regularized Quasi-Newton Method

In this chapter, we present the proposed regularized quasi-Newton method that is robust against inexact function values. This chapter is of our main contribution.

3.1 Problem Setting and Preliminaries

3.1.1 Problem Setting

We consider optimization problems in which exact objective function values are unavailable, following the recent literature on error-aware optimization [6, 11, 12, 24, 50]. When both objective values and gradients are heavily corrupted by noise, reliable optimization cannot be guaranteed. Thus, meaningful optimization requires that at least one source of information be sufficiently accurate. This work focuses on problems arising from finite-precision computations or inexact evaluations, where objective function evaluations are subject to non-negligible errors while gradient information can be computed with relatively high accuracy. Settings in which objective values are accurate but gradients are noisy, such as derivative-free optimization or stochastic approximation methods [7, 53], are beyond the scope of this paper.

In the following, we formalize the assumptions on function and gradient evaluations. We assume that only an error contaminated function $\bar{f}(x)$ is accessible, satisfying the hybrid absolute-relative error model [29, 30]:

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f \max(1, |f(x)|), \quad (3.1)$$

where $0 \leq \varepsilon_f < 1$ is an error rate. This model captures the typical behavior of rounding errors, such as in IEEE 754 floating-point arithmetic. The error is predominantly relative when $|f(x)|$ is large and effectively absolute when $|f(x)|$ is small. Closely related models are also employed in practical stopping criteria [56].

We next state assumptions regarding gradient evaluations. In practice, termination is typically based on a gradient tolerance $\varepsilon_{\text{gtol}}$, and considers $\|\nabla \bar{f}(x)\| \leq \varepsilon_{\text{gtol}}$ as a stopping criterion. In our numerical experiments, following common implementations [56], we use the infinity norm for the gradient, though other norms could equally be employed. If the gradient noise were comparable to or larger than $\varepsilon_{\text{gtol}}$, the stopping criterion could not be reliably satisfied, no matter how close x is to a stationary point. Therefore, $\varepsilon_{\text{gtol}}$ must be chosen sufficiently large relative to the gradient noise level. Accordingly, it is natural to assume that the error in gradient evaluations is small compared to the prescribed tolerance, namely,

$$\|\nabla f(x) - \nabla \bar{f}(x)\| \ll \varepsilon_{\text{gtol}}.$$

Since $\|\nabla \bar{f}(x)\| > \varepsilon_{\text{tol}}$ holds prior to termination, the gradient noise is also negligible compared to the gradient norm itself. Consequently, in our theoretical analysis, in order to isolate the effect of inexact function evaluations, we assume exact gradients:

$$\|\nabla f(x) - \nabla \bar{f}(x)\| = 0. \quad (3.2)$$

The assumption in eq. (3.2) serves as a theoretical modeling choice rather than a claim about practical implementations. Our numerical experiments in section 3.5 also test the case where gradients are contaminated by noise, demonstrating the practical effectiveness of the proposed method even when eq. (3.2) is violated.

3.1.2 Regularized Quasi-Newton Methods

In classical Newton and quasi-Newton methods, line search techniques are commonly used in practice [56]. An alternative approach is to add regularization terms to the Hessian approximation. Recent years have seen renewed interest in regularized Newton-type methods, including cubic regularization schemes [43, 48, 57]. In this work, we focus on quadratic regularization [16, 33, 36, 52, 54, 55, 64].

Let $g_k := \nabla f(x_k)$ and let $B_k \in \mathbb{R}^{n \times n}$ denote an approximate Hessian. In line-search-based quasi-Newton methods, the update is given by

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -B_k^{-1} g_k,$$

where α_k is determined by a line search. In contrast, regularized quasi-Newton methods compute the step as

$$x_{k+1} = x_k + d_k, \quad d_k = -(B_k + \mu_k I)^{-1} g_k,$$

with a regularization parameter $\mu_k \geq 0$. Combining regularization with line search is also possible and has been explored [54].

A fundamental strategy for choosing μ_k in nonconvex optimization was proposed in earlier work on regularized Newton methods [52, 54, 55]. Their formulation sets

$$\mu_k = c_1 \max(0, -\lambda_{\min}(B_k)) + c_2 \|\nabla f(x_k)\|^\delta, \quad (3.3)$$

where $c_1 > 1$, $c_2 > 0$, $\delta \geq 0$ and $\lambda_{\min}(B_k)$ denotes the smallest eigenvalue of B_k . $\|\cdot\|$ is the Euclidean norm. This choice guarantees that $B_k + \mu_k I$ is positive definite, ensuring that d_k is a descent direction. The regularization strategy adopted in this paper is closely related in spirit, and we adapt it to noisy evaluation environments.

3.1.3 Objective-Function-Free Optimization

Objective-Function-Free Optimization (OFFO) refers to a class of methods that avoid explicit evaluations of the objective function and rely solely on gradient information [25, 26]. Such methods are particularly attractive when function values are unreliable due to noise. Similar adaptive trust-region approaches are also discussed [23].

Prominent examples of OFFO include adaptive gradient methods such as ADAGRAD [18] and ADAGRAD-NORM [58]. Their update rules are

$$\begin{aligned} x_{k+1}^{(i)} &= x_k^{(i)} - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k (g_j^{(i)})^2}} g_k^{(i)}, & (i = 1, 2, \dots, n) \\ x_{k+1} &= x_k - \frac{1}{\theta \sqrt{\varsigma + \sum_{j=0}^k \|g_j\|^2}} g_k, \end{aligned}$$

respectively, where $\varsigma > 0$ and $\theta > 0$ are algorithmic parameters, and $x^{(i)}$ denotes the i -th component of a vector x .

In this work, we draw inspiration from the OFFO framework to design robust regularization and scaling strategies. Unlike purely objective-function-free methods, our algorithm exploits function values whenever they are numerically reliable, while smoothly transitioning to gradient-based control when they are not. This hybrid behavior enables improved stability without sacrificing efficiency in practical optimization tasks.

3.2 Proposed Algorithm

In this section, we present the proposed algorithm for the problem setting stated in section 3.1.1. Its pseudo-code is given in Algorithm 2, where we note that the minimum of the empty set is defined as $+\infty$ in Line 3. Algorithm 3 is an auxiliary line search algorithm called in Algorithm 2.

Algorithm 2: Noise Tolerant Regularized Quasi-Newton Method

Input: $x_0 \in \mathbb{R}^n$, $k_{\max} > 0$, $0 \leq \varepsilon_f < 1$, $0 < c < 1$, $0 < \theta_{\min} \leq \theta_{\max}$ and $\varsigma > 0$

- 1 $k \leftarrow 0$, $g_0 \leftarrow \nabla f(x_0)$, $\Delta_{-1} = 0$, $K^+ \leftarrow \emptyset$
- 2 **for** $k = 0, 1, 2, \dots, k_{\max}$ **do**
- 3 **if** $\min_{0 \leq j < k} (\bar{f}(x_j) - \Delta_j - \Delta_{j-1}) \geq \bar{f}(x_k)$ **then**
- 4 $\mu_k \leftarrow 0$
- 5 **else**
- 6 $K^+ \leftarrow K^+ \cup \{k\}$
- 7 $\mu_k \leftarrow \theta \sqrt{\varsigma + \sum_{j \in K^+} \|g_j\|^2}$ with $\theta \in [\theta_{\min}, \theta_{\max}]$ chosen adaptively
- 8 $d_k \leftarrow -(B_k + \mu_k I)^{-1} g_k$ with B_k chosen properly (section 3.4.1)
- 9 Find α_k and Δ_k satisfying eq. (3.4) by Algorithm 3
- 10 $x_{k+1} \leftarrow x_k + \alpha_k d_k$, $g_{k+1} \leftarrow \nabla \bar{f}(x_{k+1})$
- 11 **end**

Algorithm 3: Backtracking Line Search with Relaxed Armijo Condition

Input: $x_k \in \mathbb{R}^n$, $d_k \in \mathbb{R}^n$, $g_k \in \mathbb{R}^n$, $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$

- 1 $\alpha_k \leftarrow 1$
- 2 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 3 **while** $\bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k < \bar{f}(x_k + \alpha_k d_k)$ **do**
- 4 Choose $\alpha_k \in [\beta_{\min} \alpha_k, \beta_{\max} \alpha_k]$ using interpolation
- 5 $\Delta_k \leftarrow \frac{2\varepsilon_f}{1-\varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k))$
- 6 **end**
- 7 **return** α_k

In the following, we explain the three main components of Algorithm 2.

Computation of the Regularized Quasi-Newton Direction (Line 8) The first component is the computation of the regularized quasi-Newton direction d_k . Given $g_k = \nabla \bar{f}(x_k) = \nabla f(x_k)$, an approximate Hessian B_k , and a regularization parameter μ_k , we compute the search direction $d_k = -(B_k + \mu_k I)^{-1} g_k$ as in section 3.1.2. For the theoretical analysis, any choice of B_k satisfying Assumption 3 stated in section 3.3.1 is admissible, which ensures B_k is positive definite. A practical choice of B_k is discussed in section 3.4.1.

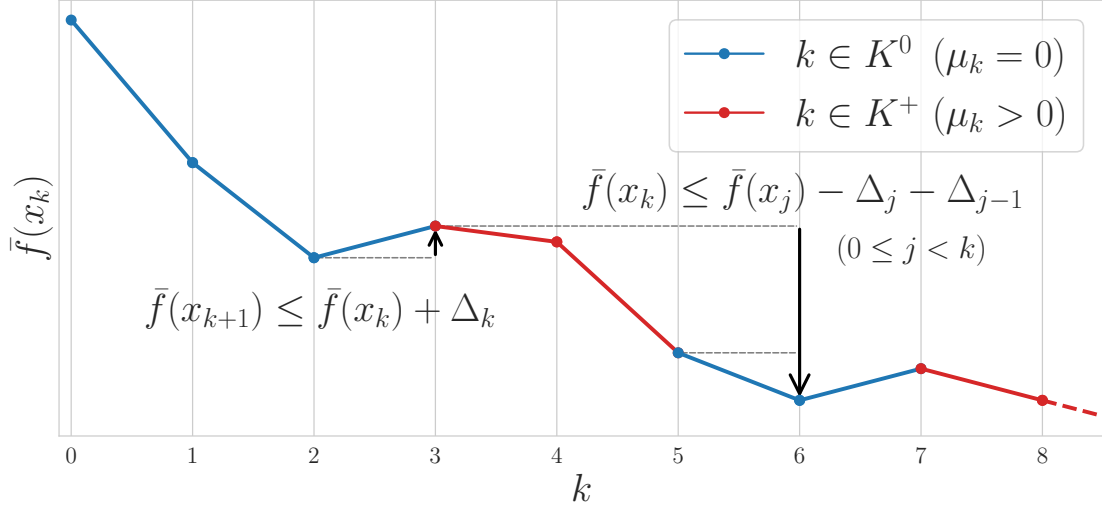


Fig. 3.1: An example of the sequence $\{\bar{f}(x_k)\}$ generated by Algorithm 2. Each iteration index k is classified into K^0 (blue) or K^+ (red). The relaxed Armijo condition eq. (3.4) ensures a temporary increase of $\bar{f}$ is at most Δ_k , such as from $k = 2$ to $k = 3$. We only set $\mu_k = 0$ when the sufficient decrease condition eq. (3.6) is satisfied over iterations in K^0 , such as at $k = 6$, preventing unbounded growth or oscillation of $\bar{f}$.

Relaxed Armijo Condition with an Error-Absorbing Term (Line 9) The second component is the computation of the step size α_k using a relaxed Armijo condition. We compute α_k satisfying the relaxed Armijo condition

$$\begin{cases} \bar{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k \geq \bar{f}(x_k + \alpha_k d_k), \\ \Delta_k := \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_k + \alpha_k d_k)) \end{cases} \quad (3.4)$$

where Δ_k is the error-absorbing term, which guarantees the existence of $\alpha_k \in (0, 1]$ even in the presence of numerical errors in $\bar{f}$. When B_k is positive definite and thus $g_k^\top d_k < 0$, eq. (3.4) ensures a temporary increase of $\bar{f}$ is at most Δ_k (Figure 3.1 illustrates this behavior). Since Δ_k depends on $-\bar{f}(x_{k+1})$ but not on $\bar{f}(x_{k+1})$, when $\bar{f}(x_{k+1}) > \bar{f}(x_k)$, the value of Δ_k is independent of how much $\bar{f}$ increases, preventing unbounded growth of Δ_k .

Adaptive Update of the Regularization Parameter (Lines 3–7) The third component is the adaptive update of the regularization parameter μ_k . A larger μ_k yields a more conservative step, while a smaller μ_k allows a more aggressive quasi-Newton step, especially when $\mu_k = 0$. In practical settings, setting $\mu_k = 0$ is preferred, since this allows us to fully exploit the quasi-Newton approximation and typically leads to practically faster convergence. Let us partition the iteration indices $K := \{0, 1, 2, \dots, k_{\max}\}$ into two sets:

$$K^0 := \{k \in K \mid \mu_k = 0\}, \quad K^+ := \{k \in K \mid \mu_k > 0\}. \quad (3.5)$$

We set $\mu_k = 0$ if and only if the following condition holds:

$$\min_{0 \leq j < k} (\bar{f}(x_j) - \Delta_j - \Delta_{j-1}) \geq \bar{f}(x_k), \quad (3.6)$$

where $\Delta_{-1} := 0$ and the minimum of the empty set is $+\infty$. This condition means that $\mu_k = 0$ is set only when a sufficient decrease in $\bar{f}$ is observed. Thereby, while $\bar{f}$ can temporarily increase, this

condition theoretically ensures that $\bar{f}$ does not grow unboundedly or oscillate indefinitely. See Figure 3.1 for an illustrative example.

Otherwise, to ensure convergence to first-order stationary points, we apply an OFFO-based update

$$\mu_k = \theta \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}, \quad (3.7)$$

as described in section 3.1.3. Since OFFO schemes do not rely on function values, this update is robust to numerical errors in $\bar{f}$.

3.3 Theoretical Analysis

In this section, we establish the global convergence properties of Algorithm 2 under the problem setting in section 3.1.1, namely, eqs. (3.1) and (3.2).

3.3.1 Assumptions

Before analyzing the convergence properties of Algorithm 2, we state three assumptions used in the analysis. The first assumption guarantees that the objective function is bounded below.

Assumption 1. *The function f is bounded below; that is, for all $x \in \mathbb{R}^n$,*

$$f(x) \geq f^* := \inf_{x \in \mathbb{R}^n} f(x) > -\infty.$$

By the triangle inequality and eq. (3.1), we have $\bar{f}(x) \geq f(x) - \varepsilon_f \max(1, |f(x)|)$. Since the function $g(t) = t - \varepsilon_f \max(1, |t|)$ is non-decreasing for $0 \leq \varepsilon_f < 1$, it follows that $\inf_x g(f(x)) = g(f^*)$. Consequently, we can also lower bound $\bar{f}$ as

$$\bar{f}(x) \geq \bar{f}^* := \inf_{x \in \mathbb{R}^n} (f(x) - \varepsilon_f \max(1, |f(x)|)) = f^* - \varepsilon_f \max(1, |f^*|) > -\infty. \quad (3.8)$$

The second assumption ensures smoothness of the objective function.

Assumption 2. *The function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is L -smooth, meaning that there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,*

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2.$$

The third assumption requires the sequence of matrices $\{B_k\}$ to be uniformly bounded and uniformly positive definite.

Assumption 3. *There exist constants $0 < m \leq 1 \leq M$ such that all B_k in Algorithm 2 satisfies*

$$mI \preceq B_k \preceq MI.$$

As we will discuss in section 3.4, the standard BFGS update with a slight modification ensures that Assumption 3 holds in practice.

3.3.2 Error Bound on Function Evaluations

We first prove a useful property of Δ_k defined in eq. (3.4). The following lemma asserts that the error between the true values difference $f(x_k) - f(x_{k+1})$ and the observed values difference $\bar{f}(x_k) - \bar{f}(x_{k+1})$ can be bounded by Δ_k .

Lemma 1. *If x_k and x_{k+1} satisfies $f(x_{k+1}) \leq f(x_k)$, it holds that*

$$f(x_k) - f(x_{k+1}) \leq \bar{f}(x_k) - \bar{f}(x_{k+1}) + \Delta_k. \quad (3.9)$$

Proof. In general (without any assumptions), we have

$$\begin{aligned} & |(f(x_k) - f(x_{k+1})) - (\bar{f}(x_k) - \bar{f}(x_{k+1}))| \\ &= |(f(x_k) - \bar{f}(x_k)) - (f(x_{k+1}) - \bar{f}(x_{k+1}))| \quad (\text{rearrangement}) \\ &\leq \varepsilon_f \max(1, |f(x_k)|) + \varepsilon_f \max(1, |f(x_{k+1})|) \quad (\text{definition in eq. (3.1)}) \\ &\leq 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) \quad (\text{max property}) \end{aligned} \quad (3.10)$$

Thus, to prove eq. (3.9), it suffices to bound eq. (3.10) by Δ_k .

By the definition of $\bar{f}$ in eq. (3.1), for any $x \in \mathbb{R}^n$ satisfying $|f(x)| > 1$, we have

$$|f(x) - \bar{f}(x)| \leq \varepsilon_f |f(x)|. \quad (3.11)$$

Since $\varepsilon_f < 1$, eq. (3.11) implies the signs of $f(x)$ and $\bar{f}(x)$ are the same. From the triangle inequality, eq. (3.11) also implies $|f(x)| - |\bar{f}(x)| \leq \varepsilon_f |f(x)|$. Since $|f(x)| > 1$ means $f(x) > 1$ or $-f(x) > 1$, we have

$$\begin{cases} f(x) \leq \frac{1}{1 - \varepsilon_f} \bar{f}(x) & \text{if } f(x) > 1, \\ -f(x) \leq \frac{1}{1 - \varepsilon_f} (-\bar{f}(x)) & \text{if } -f(x) > 1. \end{cases} \quad (3.12)$$

By using $f(x_{k+1}) \leq f(x_k)$ and the max operator as the if statement, we have

$$\begin{aligned} 2\varepsilon_f \max(1, |f(x_k)|, |f(x_{k+1})|) &= 2\varepsilon_f \max(1, f(x_k), -f(x_{k+1})) \quad (f(x_{k+1}) \leq f(x_k)) \\ &\leq \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})). \quad (\text{eq. (3.12)}) \\ &= \Delta_k. \quad (\text{eq. (3.4)}) \end{aligned}$$

Thus, we can bound eq. (3.10) by Δ_k , which completes the proof. $\square$ $\square$

3.3.3 Backtracking Stage

We next analyze the line search procedure in Algorithm 3. Recall that we assume parameters in Algorithm 3 satisfies $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$.

Proposition 13. *Under Assumptions 2 and 3, Algorithm 3 terminates if*

$$\alpha_k \leq \min \left(\frac{2(1-c)m^2}{LM}, 1 \right). \quad (3.13)$$

Proof. Under the L -smoothness of f (Assumption 2), we always have

$$\begin{aligned} f(x_k) - f(x_k + \alpha_k d_k) &\geq -\alpha_k g_k^\top d_k - \frac{L}{2} \alpha_k^2 \|d_k\|^2 \\ &= -c\alpha_k g_k^\top d_k + \alpha_k \left(-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \right). \end{aligned} \quad (3.14)$$

We first prove that if eq. (3.13) is satisfied, then the following inequality holds:

$$-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \geq 0. \quad (3.15)$$

Using $d_k = -(B_k + \mu_k I)^{-1} g_k$ (Line 8) and the spectral bounds from Assumption 3 asserting $(m + \mu_k)I \preceq B_k + \mu_k I \preceq (M + \mu_k)I$, we obtain the following two bounds:

$$-g_k^\top d_k = g_k^\top (B_k + \mu_k I)^{-1} g_k \geq \frac{1}{M + \mu_k} \|g_k\|^2, \quad (3.16)$$

$$-\|d_k\|^2 = -\|(B_k + \mu_k I)^{-1} g_k\|^2 \geq -\frac{1}{(m + \mu_k)^2} \|g_k\|^2. \quad (3.17)$$

Combining (3.16) and (3.17) with weights $(1 - c)$ and $\frac{L\alpha_k}{2}$ yields

$$-(1 - c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \geq \left(\frac{1 - c}{M + \mu_k} - \alpha_k \frac{L}{2(m + \mu_k)^2} \right) \|g_k\|^2. \quad (3.18)$$

Since eq. (3.13) gives $\alpha_k \leq \frac{2(1-c)m^2}{LM}$, eq. (3.18) can be further bounded as

$$-(1 - c)g_k^\top d_k - \frac{L\alpha_k}{2} \|d_k\|^2 \geq \frac{1 - c}{(m + \mu_k)^2} \left(\frac{(m + \mu_k)^2}{M + \mu_k} - \frac{m^2}{M} \right) \|g_k\|^2. \quad (3.19)$$

Since $0 < m < M$ as stated in Assumption 3, we have $\frac{(m + \mu_k)^2}{M + \mu_k} \geq m \cdot \frac{m + \mu_k}{M + \mu_k} \geq \frac{m^2}{M}$. Thus, using $c < 1$ and $0 \leq \mu_k$, we can establish eq. (3.15) by eq. (3.19).

Let us finish the proof. Since eq. (3.16) implies $-c\alpha_k g_k^\top d_k \geq 0$, we have

$$\begin{aligned} -c\alpha_k g_k^\top d_k &\leq f(x_k) - f(x_k + \alpha_k d_k) && \text{(eqs. (3.14) and (3.15))} \\ &\leq \bar{f}(x_k) - \bar{f}(x_k + \alpha_k d_k) + \Delta_k, && \text{(Lemma 1 and } f(x_k + \alpha_k d_k) \leq f(x_k)) \end{aligned}$$

which means the relaxed Armijo condition (eq. (3.4)) is satisfied. Thus, we can conclude that the backtracking stage terminates whenever eq. (3.13) holds. $\square$ $\square$

Let us analyze the behavior of Algorithm 3 using Proposition 13. We first bound the number of backtracking rejections as follows.

Corollary 1. *Under Assumptions 2 and 3, the number of backtracking rejections in Algorithm 3 is bounded above by*

$$\left\lceil \log \min \left(\frac{2(1-c)m^2}{LM}, 1 \right) / \log \beta_{\max} \right\rceil.$$

Proof. Let r denote the number of backtracking rejections. Initially, we have $\alpha_k = 1$ (Line 1) and each rejection multiplies α_k by at most $\beta_{\max} < 1$. Thus, after r rejections, we have $\alpha_k \leq \beta_{\max}^r$. Since the backtracking terminates once eq. (3.13) holds, $\beta_{\max}^r < \min \left(\frac{2(1-c)m^2}{LM}, 1 \right)$ is a sufficient condition for backtracking termination. Taking logarithms and rounding up gives the desired bound. $\square$ $\square$

Now, in the following, let us define

$$\bar{\alpha} := \beta_{\min} \min \left(\frac{2(1-c)m^2}{LM}, 1 \right) > 0.$$

We can lower bound the step size α_k by $\bar{\alpha}$ as follows.

Corollary 2. *Under Assumptions 2 and 3, for each iteration k of Algorithm 2, the step size α_k satisfies*

$$\alpha_k \geq \bar{\alpha}. \quad (3.20)$$

Proof. Let r denote the number of backtracking rejections. If $r = 0$, $\alpha_k = 1$ and the eq. (3.20) is obvious. Otherwise, there is a last rejected step size α'_k . By Proposition 13, $\alpha'_k > \min\left(\frac{2(1-c)m^2}{LM}, 1\right)$. Since the final α_k is chosen from $[\beta_{\min}\alpha'_k, \beta_{\max}\alpha'_k]$ as in Line 4, we have $\alpha_k \geq \beta_{\min}\alpha'_k$. Combining these inequalities yields eq. (3.20). $\square$

3.3.4 Bound for the case $\mu_k = 0$

In this subsection, we bound the sum of $\|g_k\|^2$ over $k \in K^0$. Recall that K^0 be a set of iteration indices k such that $\mu_k = 0$ in Algorithm 2 as defined in eq. (3.5). To this end, we first prove that $\bar{f}(x_k)$ and Δ_k are uniformly bounded for all $k \in K^0$, using

$$\Delta_{\max} := \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_0), -\bar{f}^*). \quad (3.21)$$

Lemma 2. *Under Assumption 1, for all $k \in K^0$ in Algorithm 2, we have*

$$\bar{f}(x_k) \leq \bar{f}(x_0), \quad \Delta_k \leq \Delta_{\max}.$$

Proof. Recall that Line 3 asserts that the following holds for $k \in K^0$:

$$\min_{0 \leq j < k} (\bar{f}(x_j) - \Delta_j - \Delta_{j-1}) \geq \bar{f}(x_k). \quad (3.22)$$

If $k > 0$, eq. (3.22) with $j = 0$ ($\Delta_{-1} = 0$ and $\Delta_0 > 0$) yields $\bar{f}(x_k) \leq \bar{f}(x_0)$. Thus, for all $k \in K^0$ including $k = 0$, we have $\bar{f}(x_k) \leq \bar{f}(x_0)$. From the lower bound of $\bar{f}$ (eq. (3.8) deduced from Assumption 1), we have $\bar{f}^* \leq \bar{f}(x_{k+1})$, which yield

$$\Delta_k = \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) \leq \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_0), -\bar{f}^*) = \Delta_{\max}.$$

$\square$

$\square$

Now, let us bound the sum of $\|g_k\|^2$ over $k \in K^0$.

Proposition 14. *Under Assumptions 1 to 3, the sum of $\|g_k\|^2$ over $k \in K^0$ satisfies*

$$\sum_{k \in K^0} \|g_k\|^2 < \frac{2M(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max})}{c\bar{\alpha}}. \quad (3.23)$$

Proof. Let us enumerate the elements of K^0 in increasing order as $K^0 = (k_i)_{i=0}^{|K^0|-1}$. The relaxed Armijo condition at iteration $k_i \in K^0$ yields

$$\begin{aligned} \bar{f}(x_{k_i}) - \bar{f}(x_{1+k_i}) + \Delta_{k_i} &\geq -c\alpha_{k_i} g_{k_i}^\top d_{k_i} && \text{(eq. (3.4))} \\ &\geq \frac{c\bar{\alpha}}{M + \mu_{k_i}} \|g_{k_i}\|^2 && \text{(Corollary 2 and eq. (3.16))} \\ &= \frac{c\bar{\alpha}}{M} \|g_{k_i}\|^2. && (\mu_{k_i} = 0 \text{ since } k_i \in K^0) \end{aligned} \quad (3.24)$$

We first consider the case of $|K^0| \geq 3$. For $0 \leq i < |K^0| - 2$, using eq. (3.22) with $j = 1 + k_i$ and $k = k_{i+2}$, and $\Delta_{1+k_i} \geq 0$, we have

$$\Delta_{k_i} \leq \bar{f}(x_{1+k_i}) - \Delta_{1+k_i} - \bar{f}(x_{k_{i+2}}) \leq \bar{f}(x_{1+k_i}) - \bar{f}(x_{k_{i+2}}). \quad (3.25)$$

Then, summing up eq. (3.24) over $0 \leq i < |K^0| - 2$ and applying eq. (3.25) yields

$$\begin{aligned} \frac{c\bar{\alpha}}{M} \sum_{i=0}^{|K^0|-3} \|g_{k_i}\|^2 &\leq \sum_{i=0}^{|K^0|-3} (\bar{f}(x_{k_i}) - \bar{f}(x_{1+k_i}) + \Delta_{k_i}) \\ &\leq \sum_{i=0}^{|K^0|-3} (\bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+2}})) \quad (\text{eq. (3.25)}) \\ &= \bar{f}(x_{k_0}) + \bar{f}(x_{k_1}) - \bar{f}(x_{k_{|K^0|-2}}) - \bar{f}(x_{k_{|K^0|-1}}). \end{aligned}$$

Thus, adding the cases $i = |K^0| - 2, |K^0| - 1$ of eq. (3.24) to the above inequality yields

$$\begin{aligned} \frac{c\bar{\alpha}}{M} \sum_{i=0}^{|K^0|-1} \|g_{k_i}\|^2 &\leq \bar{f}(x_{k_0}) + \bar{f}(x_{k_1}) - \bar{f}(x_{1+k_{|K^0|-2}}) - \bar{f}(x_{1+k_{|K^0|-1}}) + \Delta_{k_{|K^0|-2}} + \Delta_{k_{|K^0|-1}} \\ &\leq 2(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}), \end{aligned} \quad (3.26)$$

where the last inequality follows from Lemma 2 and the lower bound of $\bar{f}$ (eq. (3.8)) deduced from Assumption 1). For the case $|K^0| < 3$, we can directly obtain eq. (3.26) in a similar manner. Finally, rearranging eq. (3.26) yields the desired bound eq. (3.23), which completes the proof. $\square$

As the end of this subsection, we prove the following for the later discussion.

Lemma 3. Under Assumptions 1 to 3, the sum of Δ_k over $k \in K^0$ satisfies

$$\sum_{k \in K^0} \Delta_k \leq \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}.$$

Proof. Let us enumerate the elements of K^0 in increasing order as $K^0 = (k_i)_{i=0}^{|K^0|-1}$. Recall that $\mu_0 = 0$ and $k_0 \in K^0$, meaning that $|K^0| \geq 1$. The case $|K^0| = 1$ is obvious from $\Delta_k \leq \Delta_{\max}$. We assume $|K^0| \geq 2$ in the following. The relaxed Armijo condition (eq. (3.4)) implies $\bar{f}(x_{1+k_i}) - \Delta_{k_i} \leq \bar{f}(x_{k_i})$. Thus, for $i < |K^0| - 1$, using eq. (3.22) with $j = k_i$ and $k = k_{i+1}$, we have

$$\Delta_{k_i} \leq \bar{f}(x_{k_i}) - \Delta_{-1+k_{i+1}} - \bar{f}(x_{k_{i+1}}) \leq \bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}}). \quad (3.27)$$

Summing up eq. (3.27) over $0 \leq i < |K^0| - 1$ and adding Δ_{k_i} with $i = |K^0| - 1$ yields

$$\begin{aligned} \sum_{i=0}^{|K^0|-1} \Delta_{k_i} &\leq \sum_{i=0}^{|K^0|-2} (\bar{f}(x_{k_i}) - \bar{f}(x_{k_{i+1}})) + \Delta_{k_{|K^0|-1}} \quad (\text{eq. (3.27)}) \\ &= \bar{f}(x_{k_0}) - \bar{f}(x_{k_{|K^0|-1}}) + \Delta_{k_{|K^0|-1}} \\ &\leq \bar{f}(x_0) - \bar{f}^* + \Delta_{\max}, \quad (\text{Lemma 2 and eq. (3.8)}) \end{aligned}$$

which completes the proof. $\square$

3.3.5 Bound for the case $\mu_k > 0$

Next, we show that the sum of $\|g_k\|^2$ over $k \in K^+$ ($\mu_k > 0$) is bounded. Before proceeding to the main proof, we present two auxiliary lemmas.

The first one is about standard inequalities in the ADAGRAD-like algorithm analysis [58, Lemma 3.2], [26, Lemma 3.1]. Its proof is deferred to section A.1.

Lemma 4. *In Algorithm 2, we have*

$$\begin{aligned} \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k^2} &\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{\ell \in K^+} \|g_{k_\ell}\|^2 \right) - \ln(\varsigma) \right), \\ \sum_{k \in K^+} \frac{\|g_k\|^2}{\mu_k} &\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right). \end{aligned}$$

The second lemma is about upper-bounding a variable appearing in the following inequality. For some constants $C_1 < \infty$ and $0 < C_2 < C_3$, let $G > 0$ satisfy

$$C_1 \geq C_2 \sqrt{G} - C_3 \ln(G). \quad (3.28)$$

As we will see later, such an inequality appears when bounding $G = \varsigma + \sum_{k \in K^+} \|g_k\|^2$. Since $\sqrt{G}$ grows faster than $\ln(G)$ as $G \rightarrow \infty$, the G satisfying eq. (3.28) must be bounded above. This inequality is closely related to the Lambert W function [13]. Let e be the base of the natural logarithm. The only fact we use in this paper is that, if $x \in [-1/e, 0)$, $ye^y = x$ has two real and negative solutions, and the smaller solution is given by the second branch of Lambert W function $y^* = W_{-1}(x) < 0$ and thus $ye^y > x$ for $y < W_{-1}(x)$. In fact, a tight bound for G in eq. (3.28) can be derived as

$$G \leq \left(\frac{2C_3}{C_2} W_{-1} \left(-\frac{C_2}{2C_3} e^{-\frac{C_1}{2C_3}} \right) \right)^2 - \varsigma.$$

Refer to [26] for more details. In the following, we present a looser but simpler bound that suffices for our purpose.

Lemma 5. *For some constants $C_1 < \infty$ and $0 < C_2 < C_3$, let $G > 0$ satisfy eq. (3.28). Then, G is bounded above as*

$$G \leq \max \left(\left(\frac{2C_1}{C_2} \right)^2, G_0 \right),$$

where G_0 is

$$G_0 = \left(-\frac{4C_3}{C_2} W_{-1} \left(-\frac{C_2}{4C_3} \right) \right)^2.$$

Proof. We assume $G > G_0$ in the following. The definition of G_0 implies

$$-\frac{C_2 \sqrt{G}}{4C_3} < W_{-1} \left(-\frac{C_2}{4C_3} \right) < 0. \quad (3.29)$$

Since $-\frac{1}{e} \leq -\frac{C_2}{4C_3} < 0$ from $0 < C_2 < C_3$, the definition of W_{-1} and eq. (3.29) yield

$$-\frac{C_2 \sqrt{G}}{4C_3} e^{-\frac{C_2 \sqrt{G}}{4C_3}} > -\frac{C_2}{4C_3}.$$

This is equivalent to $\sqrt{G} < e^{\frac{C_2\sqrt{G}}{4C_3}}$, and taking logarithms gives

$$2C_3 \ln(\sqrt{G}) = C_3 \ln(G) < \frac{C_2}{2} \sqrt{G}. \quad (3.30)$$

Combining eqs. (3.28) and (3.30) yields

$$C_1 \geq C_2 \sqrt{G} - C_3 \ln(G) > C_2 \sqrt{G} - \frac{C_2}{2} \sqrt{G} = \frac{C_2}{2} \sqrt{G}.$$

Thus, we have $G \leq \left(\frac{2C_1}{C_2}\right)^2$ for all $G > G_0$, which completes the proof. $\square$ $\square$

Recall that K^+ be a set of iteration indices k such that $\mu_k > 0$ in Algorithm 2 as defined in eq. (3.5). Now, we are ready to prove the main proposition of this subsection, which bounds the sum of $\|g_k\|^2$ over $k \in K^+$.

Proposition 15. *Under Assumptions 1 to 3, there exist constants F , ς_0 , and C_0 such that the sum of $\|g_k\|^2$ over $k \in K^+$ satisfies*

$$\sum_{k \in K^+} \|g_k\|^2 \leq \left(\frac{2\theta_{\max}(F + \varsigma_0)}{\bar{\alpha}} \right)^2 + C_0. \quad (3.31)$$

Proof. The statement is trivial if K^+ is empty. Thus, we assume $K^+ \neq \emptyset$ in the following. Now, from the L -smoothness (Assumption 2), for $k \in K^+$ we have

$$\begin{aligned} f(x_k) - f(x_{k+1}) &= f(x_k) - f(x_k + \alpha_k d_k) & (x_{k+1} = x_k + \alpha_k d_k) \\ &\geq -\alpha_k g_k^\top d_k - \frac{\alpha_k^2 L}{2} \|d_k\|^2 & (\text{Assumption 2}) \\ &\geq \left(\frac{\alpha_k}{M + \mu_k} - \frac{\alpha_k^2 L}{2(m + \mu_k)^2} \right) \|g_k\|^2 & (\text{eqs. (3.16) and (3.17)}) \\ &\geq \left(\frac{\bar{\alpha}}{M + \mu_k} - \frac{L}{2(m + \mu_k)^2} \right) \|g_k\|^2. & (\text{Corollary 2 and } \alpha_k \leq 1) \end{aligned} \quad (3.32)$$

We can further bound the terms in eq. (3.32) as

$$\frac{\bar{\alpha}}{M + \mu_k} = \frac{\bar{\alpha}}{\mu_k} \frac{1}{1 + \frac{M}{\mu_k}} \geq \frac{\bar{\alpha}}{\mu_k} \left(1 - \frac{M}{\mu_k} \right) = \frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha}}{\mu_k^2}, \quad \frac{L}{2(m + \mu_k)^2} \leq \frac{L}{2\mu_k^2}. \quad (3.33)$$

Thus, applying eq. (3.33) to eq. (3.32) yields

$$f(x_k) - f(x_{k+1}) \geq \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2. \quad (3.34)$$

Summing up eq. (3.34) over $k \in K^+$ and applying Lemma 4 yield

$$\begin{aligned} &\sum_{k \in K^+} (f(x_k) - f(x_{k+1})) & (3.35) \\ &\geq \sum_{k \in K^+} \left(\frac{\bar{\alpha}}{\mu_k} - \frac{M\bar{\alpha} + L/2}{\mu_k^2} \right) \|g_k\|^2 \\ &\geq \frac{\bar{\alpha}}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - \sqrt{\varsigma} \right) - \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right) - \ln(\varsigma) \right). \end{aligned}$$

Now, let us upper bound the term in eq. (3.35). Since the indices in $\{0, 1, \dots, k_{\max}\}$ are partitioned into K^0 and K^+ , we have

$$\sum_{k \in K^+} (f(x_k) - f(x_{k+1})) = (f(x_0) - f(x_{k_{\max}})) - \sum_{k \in K^0} (f(x_k) - f(x_{k+1})). \quad (3.36)$$

From eq. (3.10) and the triangle inequality, for any $k \in K^0$, we have

$$f(x_k) - f(x_{k+1}) \geq \bar{f}(x_k) - \bar{f}(x_{k+1}) - \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, |\bar{f}(x_k)|, |\bar{f}(x_{k+1})|). \quad (3.37)$$

Using $\bar{f}(x_{k+1}) \leq \bar{f}(x_k) + \Delta_k$ and $-\bar{f}(x_k) \leq -\bar{f}(x_{k+1}) + \Delta_k$ implied by the relaxed Armijo condition (eq. (3.4)), eq. (3.37) can be further bounded as

$$\begin{aligned} f(x_k) - f(x_{k+1}) &\geq -\Delta_k - \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, |\bar{f}(x_k)|, |\bar{f}(x_{k+1})|) \\ &\geq -\Delta_k - \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k) + \Delta_k, -\bar{f}(x_{k+1}) + \Delta_k) \\ &\geq -\Delta_k - \frac{2\varepsilon_f}{1 - \varepsilon_f} \max(1, \bar{f}(x_k), -\bar{f}(x_{k+1})) - \frac{2\varepsilon_f}{1 - \varepsilon_f} \Delta_k \\ &= -\Delta_k - \left(1 + \frac{2\varepsilon_f}{1 - \varepsilon_f}\right) \Delta_k \quad (\text{Definition of } \Delta_k \text{ in eq. (3.4)}) \\ &= -\frac{2}{1 - \varepsilon_f} \Delta_k. \end{aligned} \quad (3.38)$$

Combining eqs. (3.36) and (3.38), we can upper bound eq. (3.35) as

$$\begin{aligned} \sum_{k \in K^+} (f(x_k) - f(x_{k+1})) &\leq (f(x_0) - f^*) - \sum_{k \in K^0} (f(x_k) - f(x_{k+1})) \quad (\text{eq. (3.36)}) \\ &\leq (f(x_0) - f^*) + \frac{2}{1 - \varepsilon_f} \sum_{k \in K^0} \Delta_k \quad (\text{eq. (3.38)}) \\ &\leq (f(x_0) - f^*) + \frac{2}{1 - \varepsilon_f} (\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}). \quad (\text{Lemma 3}) \end{aligned} \quad (3.39)$$

Using eq. (3.39) in eq. (3.35), we obtain

$$C_1 \geq C_2 \sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2} - C_3 \ln \left(\varsigma + \sum_{k \in K^+} \|g_k\|^2 \right)$$

where

$$\begin{aligned} C_1 &:= (f(x_0) - f^*) + \frac{2}{1 - \varepsilon_f} (\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}) + \frac{\sqrt{\varsigma}}{\theta_{\max}} - \frac{M + L/2}{\theta_{\min}^2} \ln(\varsigma), \\ C_2 &:= \frac{\bar{\alpha}}{\theta_{\max}}, \quad C_3 := \frac{M\bar{\alpha} + L/2}{\theta_{\min}^2}. \end{aligned}$$

From $1 \leq M$ and $\theta_{\min} \leq \theta_{\max}$, we have $0 < C_2 < C_3$. Thus, Lemma 5 implies

$$\varsigma + \sum_{k \in K^+} \|g_k\|^2 \leq \max \left(\left(\frac{2\theta_{\max}(F + \varsigma_0)}{\bar{\alpha}} \right)^2, G_0 \right) \quad (3.40)$$

where

$$\begin{aligned} F &:= (f(x_0) - f^*) + \frac{2}{1 - \varepsilon_f} (\bar{f}(x_0) - \bar{f}^* + \Delta_{\max}), \\ \varsigma_0 &:= \frac{\sqrt{\varsigma}}{\theta_{\max}} - \frac{M + L/2}{\theta_{\min}^2} \ln(\varsigma), \\ G_0 &:= \left(\frac{4C_3}{C_2} W_{-1} \left(-\frac{C_2}{4C_3} \right) \right)^2 = \left(\frac{4\theta_{\max}(M\bar{\alpha} + L/2)}{\theta_{\min}^2 \bar{\alpha}} W_{-1} \left(-\frac{\theta_{\min}^2 \bar{\alpha}}{4\theta_{\max}(M\bar{\alpha} + L/2)} \right) \right)^2. \end{aligned}$$

Defining $C_0 := \max(0, G_0 - \varsigma)$ yields

$$\max \left(\left(\frac{2\theta_{\max}(F + \varsigma_0)}{\bar{\alpha}} \right)^2, G_0 \right) - \varsigma \leq \left(\frac{2\theta_{\max}(F + \varsigma_0)}{\bar{\alpha}} \right)^2 + C_0. \quad (3.41)$$

Combining eqs. (3.40) and (3.41) yields eq. (3.31), which completes the proof. $\square$ $\square$

3.3.6 Global Convergence Result

Now, we establish the following global convergence result for Algorithm 2.

Theorem 3 (Global Convergence). *Under Assumptions 1 to 3 with $k > 0$, Algorithm 2 satisfies the following global convergence rate:*

$$\frac{1}{k+1} \sum_{j=0}^k \|g_j\|^2 = \mathcal{O}(1/k).$$

Proof. Using Propositions 14 and 15, we have

$$\begin{aligned} \sum_{j=0}^k \|g_j\|^2 &= \sum_{k \in K^0} \|g_k\|^2 + \sum_{k \in K^+} \|g_k\|^2 \\ &< \frac{2M(\bar{f}(x_0) - \bar{f}^* + \Delta_{\max})}{c\bar{\alpha}} + \left(\frac{2\theta_{\max}(F + \varsigma_0)}{\bar{\alpha}} \right)^2 + C_0, \end{aligned} \quad (3.42)$$

where the right-hand side is constant with respect to k . By dividing both sides by $k+1$, we complete the proof. $\square$ $\square$

Theorem 3 is a standard convergence guarantee for first-order optimization algorithms applied to L -smooth nonconvex functions. Furthermore, this result is equivalent to the following iteration complexity:

$$\min \{k \mid \|\nabla f(x_k)\| < \varepsilon\} = \mathcal{O}(1/\varepsilon^2).$$

We interpret the three terms in eq. (3.42) in light of classical optimization results.

- The first term is an analogue of the classical gradient descent bound for an error-contaminated function $\bar{f}$. Indeed, for gradient descent with fixed step size $\alpha = 1/L$, the sum of squared gradients is bounded as

$$\sum_{j=0}^k \|g_j\|^2 \leq 2L(f(x_0) - f^*) = \frac{2(f(x_0) - f^*)}{\alpha}.$$

- The second term mirrors the convergence property of ADAGRAD-NORM [58], which is scaling as $(f(x_0) - f^*)^2$. This follows from the fact that F contains the initial optimality gaps $f(x_0) - f^*$ and $\bar{f}(x_0) - \bar{f}^*$ (see the proof of Proposition 15), and the term is quadratic in F .
- The third term is C_0 , which depends only on the algorithmic parameters and the smoothness constant L , acting as a constant offset.

3.4 Practical Choices of Algorithmic Variables

In sections 3.2 and 3.3, we introduced Algorithm 2 and established its convergence properties. In this section, we discuss practical choices of algorithmic variables and parameters that lead to fast and stable performance in practice. The quantities to be discussed here are the matrices B_k , the step sizes α_k , and the regularization parameters μ_k .

3.4.1 Approximate Hessian B_k

We first discuss the construction of the matrices B_k used in Line 8 of Algorithm 2. As mentioned in section 1.1, we employ an L-BFGS method to form B_k . The purpose of this subsection is to show that B_k can satisfy Assumption 3, the uniform spectral bound, with the modification and selection of curvature pairs.

Construction of B_k Before stating concrete construction of B_k , we recall the L-BFGS method and the curvature pair notation [8, 37, 44]. For an iteration k , we define

$$s_k := x_{k+1} - x_k, \quad y_k := g_{k+1} - g_k \quad (3.43)$$

where g_k and g_{k+1} are the gradients at x_k and x_{k+1} , respectively. Starting from an initial matrix, an approximate Hessian is obtained by successively applying BFGS updates to a sequence of curvature pairs. Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a collection of curvature pairs, where k_i denotes the iteration index at which the i -th pair was generated. Starting from the initial matrix γI with $\gamma = \|y_{k_0}\|^2 / y_{k_0}^\top s_{k_0}$, we define $\text{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1})$ as the matrix obtained by sequentially applying the BFGS update using these pairs in the given order. Here, p denotes the maximum number of stored curvature pairs and is a fixed small integer in the L-BFGS method. When fewer than p curvature pairs are available, the BFGS update is applied using all currently available pairs.

Now, we provide a sufficient condition for Assumption 3 by the following proposition, which is a variant of existing work [20, Theorem 5.5] [22, Lemma 1]. Its proof is deferred to section A.2.

Proposition 16. *Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a fixed number of curvature pairs with $s_{k_i} \neq 0$. Assume that there exist constants $0 < \lambda \leq \Lambda$ such that for all $(s, y) \in \{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$,*

$$y^\top s \geq \frac{1}{\Lambda} \|y\|^2, \quad (3.44)$$

$$y^\top s \geq \lambda \|s\|^2. \quad (3.45)$$

Define the condition number $\kappa := \Lambda/\lambda$. Then there exist $0 < m < M$ such that

$$mI \preceq \text{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}) \preceq MI, \quad (3.46)$$

where

$$M := \gamma + p\Lambda, \quad m := \left((1 + \sqrt{\kappa})^{2p} \left(\gamma + \frac{1}{\lambda(2\sqrt{\kappa} + \kappa)} \right) \right)^{-1}.$$

Proposition 16 shows that, whenever eqs. (3.44) and (3.45) hold for the stored pairs, the resulting BFGS matrix satisfies Assumption 3 as expressed in eq. (3.46).

Based on Proposition 16, we basically adopt the following rule for forming B_k :

$$B_k = \text{BFGS}(\{(s_{k_i}, \bar{y}_{k_i} := \theta_i^{\text{damp}} y_{k_i} + (1 - \theta_i^{\text{damp}}) B_{k-1} s_{k_i})\}), \quad (3.47)$$

where $\theta_i^{\text{damp}} \in [0, 1]$ is determined using damped BFGS update [38, 46] and the indices $\{k_i\}_{i=0}^{p-1}$ are selected so that the modified pairs $(s_{k_i}, \bar{y}_{k_i})$ satisfy eqs. (3.44) and (3.45) for some constants λ, Λ . The damped BFGS is a classical technique to ensure $\bar{y}_{k_i}^\top s_{k_i} > 0$.

Since we do not employ the Wolfe conditions in the line search, this damping technique plays a central role in ensuring the positive definiteness of B_k . The selection of curvature pairs shares the same spirit as the cautious update [35, 39, 40], which only uses pairs satisfying certain curvature conditions. By Proposition 16, B_k satisfies the uniform spectral bound in Assumption 3. This confirms that the practical construction of B_k is consistent with the theoretical requirements.

Remarks for Practical Implementation Having established that the proposed construction of B_k satisfies Assumption 3, we now present several remarks on its practical implementation. We begin by describing how to compute the search direction

$$d_k = -(B_k + \mu_k I)^{-1} g_k \quad (3.48)$$

with B_k in eq. (3.47). When $\mu_k = 0$, the direction can be computed efficiently by the standard L-BFGS two-loop recursion [37, 44]. For the case $\mu_k > 0$, regularized limited-memory schemes may in principle be employed [33]. Alternatively, one may adopt the simple and robust approximation mentioned in the same reference. Specifically, we approximate B_k in eq. (3.47) by

$$B_k \approx \text{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1}) - \mu_k I, \quad (3.49)$$

so that $B_k + \mu_k I \approx \text{BFGS}(\{s_{k_i}, \bar{y}_{k_i} + \mu_k s_{k_i}\}_{i=0}^{p-1})$. Its inverse can then be computed using the standard two-loop recursion. Although this approximation is inexact, it has been reported to yield nearly identical practical performance while remaining comparatively simple to implement [33]. We adopt this approximation in this work.

Separately, there exist function-based modified secant conditions [2, 28, 61, 65] that adjust the secant equation using available function values and gradients when those quantities are deemed reliable. Such adjustments aim to improve practical convergence behavior. We partially adopt these function-based modifications following prior work [62, 66]. The suffix -MS in section 3.5 indicates the use of this technique. Because these modifications are well established, we omit low-level implementation details here. Further specifics are available in the cited references and in our open-source implementation*¹.

3.4.2 Step Size α_k

We next discuss the adjustment of the step size α_k in Line 4 of Algorithm 3.

As indicated in the pseudo-code, we choose α_k using interpolation of the objective function. Moré–Thuente line searches [41] are widely used in quasi-Newton methods [56], and they typically employ cubic or quadratic interpolation to select trial step sizes. In our implementation, we use essentially the same procedure with a slight modification, namely, clipping the trial step size to the interval $[\beta_{\min} \alpha_k, \beta_{\max} \alpha_k]$. For all numerical experiments, we set the Armijo parameter to $c = 10^{-4}$ and choose the interpolation bounds as $\beta_{\min} = 1/16$ and $\beta_{\max} = 15/16$.

Furthermore, when $\mu_k > 0$ and $\alpha_k = 1$, we introduce a one-time adjustment of the trial step size using gradient information. Let d_k be the search direction and g_k and g_{try} the gradients at the current and trial points. If the directional derivative along d_k changes sign, $d_k^\top g_k < 0$ and

$d_k^\top g_{\text{try}} > 0$, and the trial gradient is sufficiently aligned with d_k , namely $d_k^\top g_{\text{try}} > 0.5\|d_k\|\|g_{\text{try}}\|$, we rescale α_k once by a secant-like factor,

$$\alpha_k \leftarrow \text{clip}\left(\alpha_k \frac{-d_k^\top g_k}{d_k^\top g_{\text{try}} - d_k^\top g_k}, \beta_{\min} \alpha_k, \beta_{\max} \alpha_k\right),$$

where clip means clipping to the specified interval. When objective values are unreliable and backtracking and interpolation do not occur, this correction effectively refines the step size. Since this adjustment is at most once per iteration, the theoretical results in section 3.3 remain valid.

3.4.3 Regularization Parameter μ_k

We finally discuss the adjustment of the regularization parameter μ_k in Line 7 of Algorithm 2.

In our implementation, we choose the regularization parameter as

$$\mu_k = \text{clip}\left(\frac{1}{10}\|g_k\|, \frac{1}{100}G_k, G_k\right) \text{ where } G_k := \sqrt{\varsigma + \sum_{j \in K, j \leq k} \|g_j\|^2},$$

which is consistent with the general form (eq. (3.7)) with an adaptive factor $\theta_k \in [10^{-2}, 1]$. In practice, we may set $\varsigma = 0$ as long as the initial gradient is nonzero since ς is mainly introduced for avoiding zero division, but to make it consistent with the theory, we set $\varsigma = 10^{-10}$ in all experiments.

When B_k is a good approximation of the Hessian, the regularization parameter μ_k should be small for fast convergence. Since the quantity G_k is strictly increasing with k , it is natural in practice to decrease the effective regularization when sufficient progress in the objective function is achieved. When the corrected objective value decreases by more than 1, we restart the accumulation by resetting K^+ to the empty set and G_k to ς . As long as the number of such restarts is finite, the overall convergence analysis in section 3.3 remains valid. Since these choices are heuristic, there may be room for further improvement. Especially, further increasing μ_k stabilizes the method when the objective function values are highly noisy, and decreasing μ_k more aggressively may accelerate convergence when the objective function is accurate.

3.5 Numerical Experiments

3.5.1 Methods

We compared the following optimization algorithms:

- (Ours): Our proposed regularized quasi-Newton method
 - We used Algorithm Algorithms 2 and 3 with the details in section 3.4.
- (Line): Line-search L-BFGS method
 - Our Python implementation ported from LBFGSpp [45].
 - The step size was determined using the Moré–Thuente line search, closely resembling SciPy’s implementation.
- (SciPy): SciPy’s L-BFGS-B method [56]
 - An established L-BFGS method implemented in C and called from Python.
 - We used the default parameters, with the exception that `ftol` was set to 0 to avoid premature termination.
- (Reg): Regularized L-BFGS method [33]
 - A quadratic regularizationbased quasi-Newton method that does not rely on line search and is conceptually closer to trust-region methods.

explanation	min abs value	relative error	ε_f
64-bit double precision	2.23×10^{-308}	$2^{-52} \approx 2.22 \times 10^{-16}$	2.22×10^{-9}
32-bit single precision	1.18×10^{-38}	$2^{-23} \approx 1.19 \times 10^{-7}$	1.19×10^{-3}
16-bit half precision	6.10×10^{-5}	$2^{-10} \approx 9.77 \times 10^{-4}$	9.77×10^{-3}

Table 3.1: The “min abs value” is the smallest positive normalized number, the “relative error” is the maximum relative rounding error, and ε_f is the parameter in (3.1) used in our experiments.

- We wrapped the function `regLBFGS` using `solveNonmonotone`.
- (NTQN): Noise-tolerant quasi-Newton method [53, 60]
 - A method designed to accommodate inexact gradient information, as mentioned in section 3.1.1.
 - We mainly used the default parameter settings, and slightly modified the stopping criteria to terminate at the desired gradient norm tolerance.

In the following, each method is referred to by the name given in parentheses. For (Ours) and (Line), we additionally tested variants incorporating the modified secant condition described in section 3.4.1, which are denoted by (Ours-MS) and (Line-MS), respectively. All these methods are L-BFGS-based, and we set the number of stored curvature pairs to 10, the default value in SciPy’s L-BFGS-B implementation.

All experiments were conducted on a Microsoft Windows 11 Home system (version 10.0, build 26100) equipped with a 13th Gen Intel® Core™ i7-1360P CPU, featuring 12 physical cores and 16 logical processors. All computations were performed using the CPU only.

3.5.2 Experimental Results

Test Problems

We evaluated all algorithms on unconstrained problems from the CUTEst collection [21], implemented in Python using the PyCUTEst interface [19]. All problems were initialized with the default starting points provided by the library. Following the previous work [33], we excluded 17 problems where either the initial point is already stationary or all algorithms are known to fail. This left 220 test problems. To simulate different numerical environments, we tested the following four settings:

- Artificial noise with absolute noise level of 10^{-3} at 64-bit precision (220 problems, $\varepsilon_f = 10^{-2}$)
- 64-bit floating-point precision (220 problems, $\varepsilon_f = 2.22 \times 10^{-9}$)
- 32-bit floating-point precision (220 problems, $\varepsilon_f = 1.19 \times 10^{-2}$)
- 16-bit floating-point precision (152 problems, $\varepsilon_f = 9.77 \times 10^{-1}$)

Each setting uses a different error rate ε_f in the relative error model (eq. (3.1)), as summarized in Table 3.1. The table relates the machine epsilon (maximum relative rounding error) for each floating-point precision to the ε_f value used in our experiments. We set ε_f equal to or larger than the machine epsilon to account for additional uncertainties in function and gradient evaluations.

In the reduced precision settings, to simulate lower numerical precision, we first casted the input vector x to the target lower precision, then computed $\bar{f}(x)$ and $\nabla \bar{f}(x)$ using the lower-precision input. Although the internal computations of CUTEst remain in 64-bit precision, the casting of the input vector effectively simulated the impact of reduced precision on function and gradient evaluations. In the 16-bit precision setting, we excluded problems where the initial gradient norm contains NaN or INF values after casting, resulting in a total of 152 problems.

In the artificial noise setting, we added uniform random noise sampled from $[-10^{-3}, 10^{-3}]$ to

both the objective function values and each component of the gradient vectors independently, i.e.,

$$\bar{f}(x) = f(x) + \text{unif}(-10^{-3}, 10^{-3}), \quad \nabla \bar{f}(x) = \nabla f(x) + \text{unif}(-10^{-3}, 10^{-3}) \mathbb{1},$$

where $\text{unif}(a, b)$ denotes a uniform random variable in the interval $[a, b]$ and $\mathbb{1}$ is the vector of all ones. This setting is the same as the one in [53].

Performance Profiles

We used performance profiles [17] to compare the algorithms. Let P denote the set of test problems and S the set of solvers. For each problem $p \in P$ and solver $s \in S$, let $t_{p,s} > 0$ denote the number of function and gradient evaluations required to reach a specified gradient norm tolerance $\varepsilon_{\text{gtol}}$. Note that $t_{p,s}$ is not time in this experiment but the number of evaluations. When a solver fails to converge within the gradient tolerance or exceeds a maximum number of evaluations, we set $t_{p,s} = +\infty$. Then, the performance ratio $r_{p,s}$ is defined as

$$r_{p,s} = \frac{t_{p,s}}{\min_{s' \in S} t_{p,s'}}.$$

For a given threshold $\tau \geq 1$, the performance profile plots the proportion of problems for which $r_{p,s} \leq \tau$, which means that the y -coordinate for solver s at x -coordinate τ is given by

$$\rho_s(\tau) = \frac{1}{|P|} |\{p \in P \mid r_{p,s} \leq \tau\}|.$$

A curve that lies above others indicates better overall performance.

Results with Artificial Noise

We first present the results in the artificial noise setting described in section 3.5.2. Figure 3.2 presents the performance profile with $\varepsilon_f = 10^{-2}$. In this regime, the proposed method shows superior robustness compared to other quasi-Newton methods. Notably, (Ours) and (Ours-MS) outperform (NTQN), which is specifically designed to handle noisy gradients.

Results at Different Precision Levels

We next present performance profiles for each floating-point precision (64-bit, 32-bit, and 16-bit). We evaluated each method at three gradient norm tolerance levels (10^{-3} , 10^{-4} , and 10^{-5}), i.e., $\varepsilon_{\text{gtol}} \in \{10^{-3}, 10^{-4}, 10^{-5}\}$. These tolerance levels represent different optimization regimes, from moderate accuracy to high precision requirements. Figure 3.3 shows the comprehensive performance profiles across all three precision levels. Our method (Ours) demonstrates competitive or superior performance compared to existing methods in the standard 64-bit setting. The modified secant equation variant (Ours-MS) further improves performance. As numerical precision degrades from 64-bit to 32-bit and further to 16-bit, our regularizationbased approach demonstrates improved relative performance. The proposed method maintains stability even when traditional line-search methods encounter difficulties with reduced precision arithmetic.

3.5.3 Computational Time

In this subsection, we present an experiment to compare the per-iteration computational cost of each method. Since the previous experiments focused on the number of function and gradient evaluations, they did not directly reflect the computational overhead of each algorithm. Here, we measured the total execution time required to perform a fixed number of iterations on a simple problem. We conducted experiments on the ill-conditioned quadratic problem

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \quad \frac{1}{2} \sum_{i=1}^n ix_i^2$$

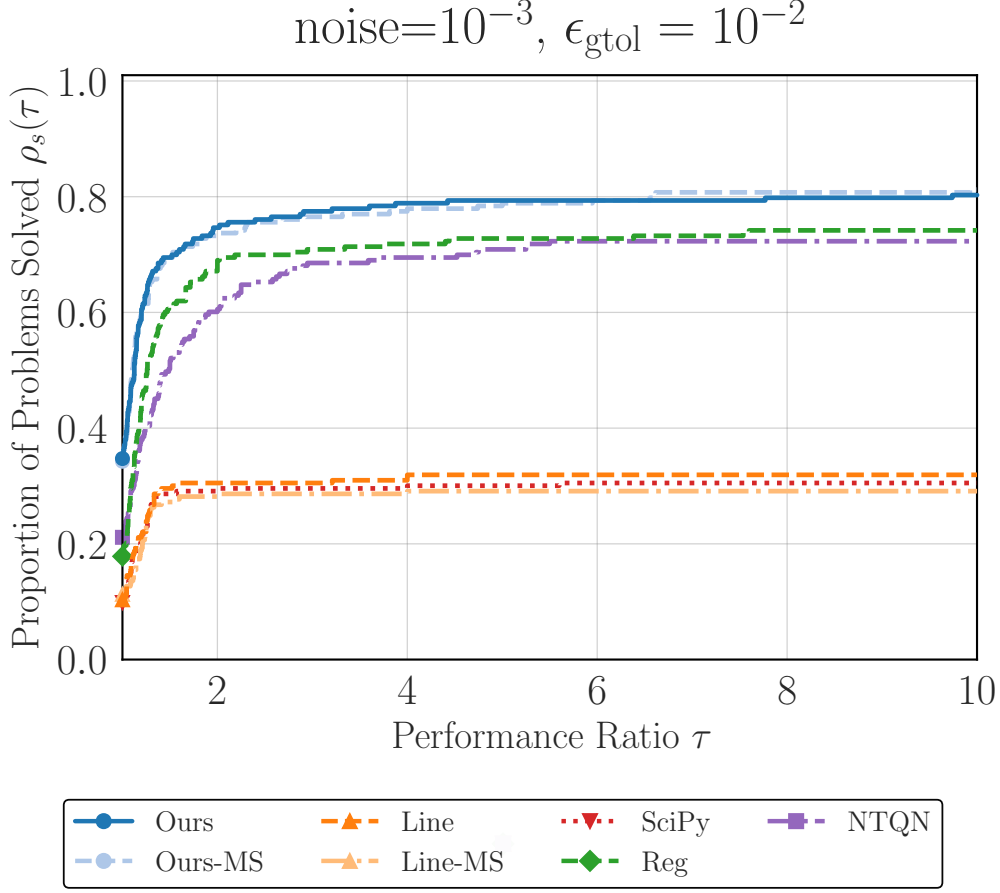


Fig. 3.2: Performance profile for artificial noise setting (level of 10^{-3}) and $\epsilon_{\text{gtol}} = 10^{-2}$.

with $n = 10,000$, and measured the total execution time. Timing data were collected after three warm-up runs, followed by 100 recorded trials. The initial point x_0 was chosen as the all-ones vector. We used memory size $m = 10$ and maximum iterations $k_{\max} = 100$, and we did not set any stopping criteria other than reaching $k_{\max}$. The simplicity of the optimization problem and the small value of $k_{\max}$ are well suited to the purpose of this experiment, as they ensure that the total runtime is dominated by algorithmic overhead rather than function or gradient computation.

The results are summarized in Table 3.2 and Figure 3.4. In Table 3.2, we report the total number of oracle calls (calls of $\bar{f}$ and $\nabla \bar{f}$) and final objective values $f(x_{k_{\max}})$. We only report the unique values per method since both quantities were identical across runs for each method. The comparable values in the table indicate that the comparison of execution times is fair and not affected by differences in convergence behavior. In Figure 3.4, we report the execution time statistics, and we can observe that our proposed methods (Ours) and (Ours-MS) exhibit competitive performance compared to other methods. Although the (SciPy) calls optimized C routines, it incurs associated dispatch overhead, producing a larger mean time. The (Reg) method explicitly computes $-(B_k + \mu_k I)^{-1} g_k$, which is numerically attractive but entails extra per-iteration cost. In contrast, our proposed methods employ the simplified strategy described in section 3.4, which reduces per-iteration overhead while preserving performance. This design accounts for their competitive mean execution times.

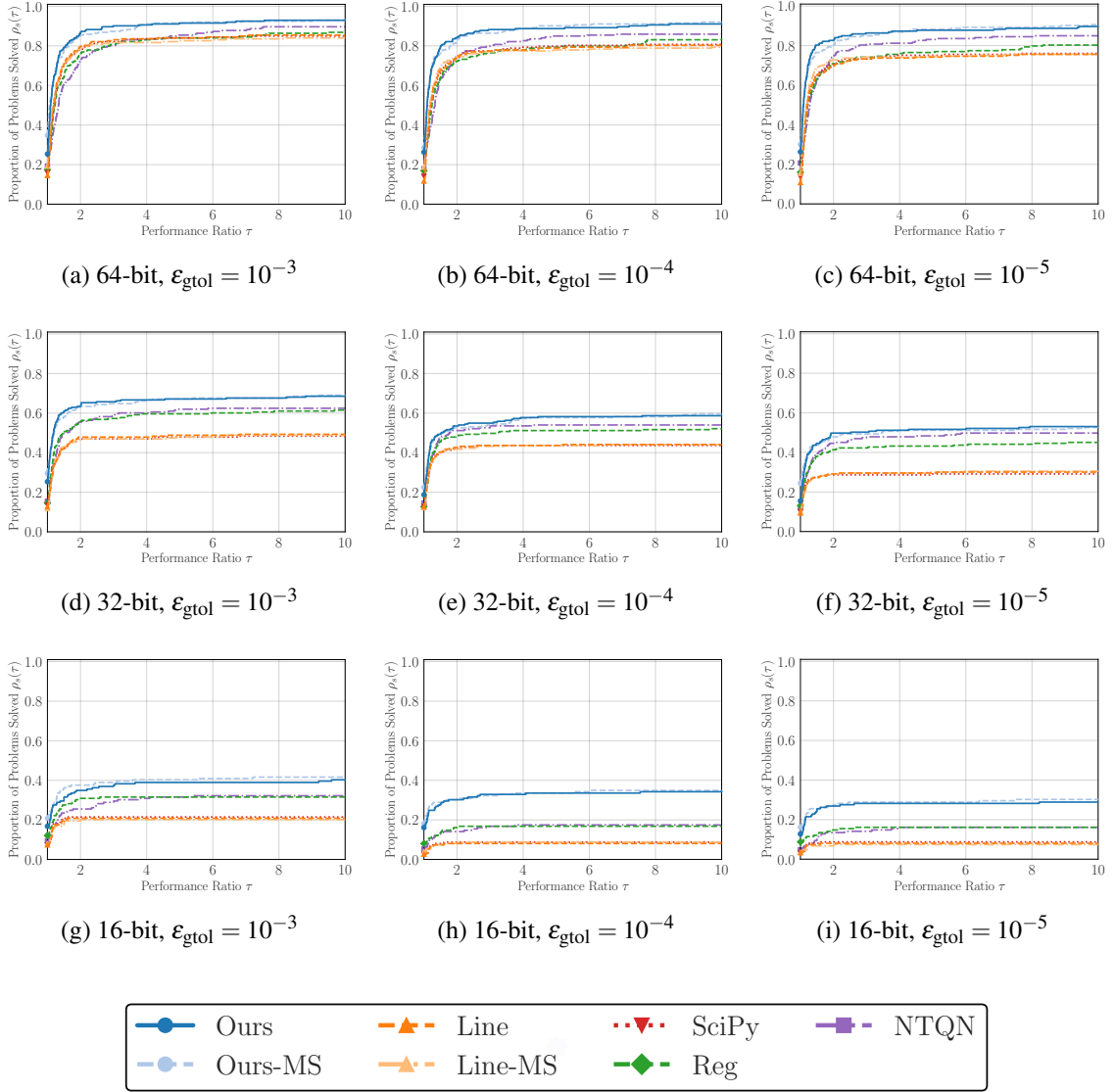


Fig. 3.3: Performance profiles across different floating-point precisions (64, 32, and 16-bit) and tolerance levels. Each row corresponds to a different precision level, demonstrating the robustness of the proposed method as numerical precision degrades.

3.6 Conclusion

In this work, we proposed a regularized quasi-Newton method for noisy nonconvex optimization and established its global convergence properties. We also demonstrated the method's effectiveness through numerical experiments on standard benchmark problems under various noise and precision settings. The results indicate that our method is robust and practically efficient, particularly in noisy or low-precision environments where traditional line-search methods may struggle.

Future work includes the investigation of local convergence properties, the extension to constrained optimization, and the application to practical problems in machine learning and scientific computing. Additionally, exploring adaptive strategies for selecting algorithmic parameters based on problem characteristics could further enhance the method's performance.

	Line	Line-MS	Ours	Ours-MS	NTQN	SciPy	Reg
# Calls	212	212	202	202	219	212	206
$\bar{f}(x_{k_{\max}})$	1.24	1.24	1.34	1.34	1.43	1.24	1.47

Table 3.2: The number of oracle calls and final observed objective values for the experiment in section 3.5.3.

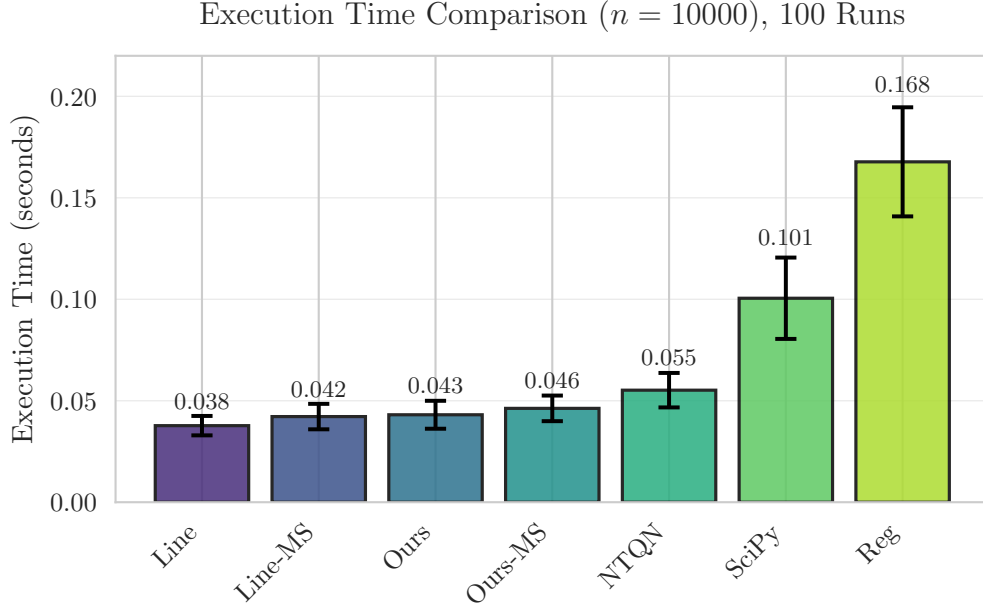


Fig. 3.4: Mean execution times for the experiment in section 3.5.3. This plot roughly indicates the extra computational overhead of each method. Error bars represent one standard deviation over 100 runs.

Explicitly analyzing the impact of inexact gradient evaluations can also provide valuable insights into the method's robustness and guide further enhancements.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Professor Akiko Takeda, and Project Research Associate Naoki Marumo for their extensive guidance on the direction and content of the research.

Bibliography

- [1] Babaie-Kafaki S (2011) A modified BFGS algorithm based on a hybrid secant equation. *Science China Mathematics* 54(9):2019–2036, DOI 10.1007/s11425-011-4232-7
- [2] Babaie-Kafaki S, Ghanbari R, Mahdavi-Amiri N (2010) Two new conjugate gradient methods based on modified secant equations. *Journal of Computational and Applied Mathematics* 234(5):1374–1386, DOI 10.1016/j.cam.2010.01.052
- [3] Baillon JB, Haddad G (1977) Quelques propriétés des opérateurs angle-bornés et cycliquement monotones. *Israel Journal of Mathematics* 26(2):137–150, DOI 10.1007/BF03007664
- [4] Barzilai J, Borwein JM (1988) Two-Point Step Size Gradient Methods. *IMA Journal of Numerical Analysis* 8(1):141–148, DOI 10.1093/imanum/8.1.141
- [5] Bauschke HH, Combettes PL (2009) The Baillon-Haddad Theorem Revisited. DOI 10.48550/arXiv.0906.0807, [0906.0807](#)
- [6] Bellavia S, Gurioli G, Morini B, Toint PL (2021) The Impact of Noise on Evaluation Complexity: The Deterministic Trust-Region Case. DOI 10.48550/arXiv.2104.02519, [2104.02519](#)
- [7] Berahas AS, Cao L, Choromanski K, Scheinberg K (2021) A Theoretical and Empirical Comparison of Gradient Approximations in Derivative-Free Optimization. DOI 10.48550/arXiv.1905.01332, [1905.01332](#)
- [8] Berahas AS, Curtis FE, Zhou B (2022) Limited-memory BFGS with displacement aggregation. *Mathematical Programming* 194(1):121–157, DOI 10.1007/s10107-021-01621-6
- [9] Byrd RH, Nocedal J (1989) A Tool for the Analysis of Quasi-Newton Methods with Application to Unconstrained Minimization. *SIAM Journal on Numerical Analysis* 26(3):727–739, DOI 10.1137/0726042
- [10] Caffisch RE (1998) Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica* 7:1–49, DOI 10.1017/S0962492900002804
- [11] Carter RG (1993) Numerical Experience with a Class of Algorithms for Nonlinear Optimization Using Inexact Function and Gradient Information. *SIAM Journal on Scientific Computing* 14(2):368–388, DOI 10.1137/0914023
- [12] Clancy RJ, Menickelly M, Hückelheim J, Hovland P, Nalluri P, Gjini R (2022) TROPHY: Trust Region Optimization Using a Precision Hierarchy. In: *Computational Science – ICCS 2022*, Springer, Cham, pp 445–459, DOI 10.1007/978-3-031-08751-6_32
- [13] Corless RM, Gonnet GH, Hare DEG, Jeffrey DJ, Knuth DE (1996) On the LambertW function. *Advances in Computational Mathematics* 5(1):329–359, DOI 10.1007/BF02124750
- [14] Dennis JE Jr, Moré JJ (1977) Quasi-Newton Methods, Motivation and Theory. *SIAM Review* 19(1):46–89, DOI 10.1137/1019005
- [15] Doikov N (2021) New second-order and tensor methods in convex optimization. PhD thesis, Université catholique de Louvain
- [16] Doikov N, Mishchenko K, Nesterov Y (2024) Super-Universal Regularized Newton Method. *SIAM Journal on Optimization* 34(1):27–56, DOI 10.1137/22M1519444
- [17] Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. *Mathematical Programming* 91(2):201–213, DOI 10.1007/s101070100263
- [18] Duchi J, Hazan E, Singer Y (2011) Adaptive Subgradient Methods for Online Learning and

- Stochastic Optimization. *Journal of Machine Learning Research* 12(61):2121–2159
- [19] Fowkes J, Roberts L, Bűrmen Á (2022) PyCUTEst: An open source Python package of optimization test problems. *Journal of Open Source Software* 7(78):4377, DOI 10.21105/joss.04377
 - [20] Gao W, Goldfarb D (2018) Quasi-Newton Methods: Superlinear Convergence Without Line Searches for Self-Concordant Functions. DOI 10.48550/arXiv.1612.06965, [1612.06965](https://arxiv.org/abs/1612.06965)
 - [21] Gould NI, Orban D, Toint PL (2015) CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. *Comput Optim Appl* 60(3):545–557, DOI 10.1007/s10589-014-9687-3
 - [22] Gower RM, Goldfarb D, Richtárik P (2016) Stochastic Block BFGS: Squeezing More Curvature out of Data. DOI 10.48550/arXiv.1603.09649, [1603.09649](https://arxiv.org/abs/1603.09649)
 - [23] Grapiglia GN, Stella GFD (2022) An adaptive trust-region method without function evaluations. *Computational Optimization and Applications* 82(1):31–60, DOI 10.1007/s10589-022-00356-0
 - [24] Gratton S, Toint PL (2020) A note on solving nonlinear optimization problems in variable precision. *Computational Optimization and Applications* 76(3):917–933, DOI 10.1007/s10589-020-00190-2
 - [25] Gratton S, Toint PL (2023) OFFO minimization algorithms for second-order optimality and their complexity. *Computational Optimization and Applications* 84(2):573–607, DOI 10.1007/s10589-022-00435-2
 - [26] Gratton S, Jerad S, Toint PhL (2024) Complexity of a class of first-order objective-function-free optimization algorithms. *Optimization Methods and Software* 0(0):1–31, DOI 10.1080/10556788.2023.2296431, <https://doi.org/10.1080/10556788.2023.2296431>
 - [27] Haelterman R (2009) Analytical study of the Least Squares Quasi-Newton method for interaction problems. PhD thesis, 2009.
 - [28] Hassan BA, Moghrabi IAR (2023) A modified secant equation quasi-Newton method for unconstrained optimization. *Journal of Applied Mathematics and Computing* 69(1):451–464, DOI 10.1007/s12190-022-01750-x
 - [29] Higham NJ (2002) Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, DOI 10.1137/1.9780898718027
 - [30] Izycheva A, Darulova E (2017) On sound relative error bounds for floating-point arithmetic. In: *Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design*, FMCAD Inc, Austin, Texas, Fmcad '17, pp 15–22
 - [31] Kanamori T, Ohara A (2010) A Bregman Extension of quasi-Newton updates I: An Information Geometrical framework. DOI 10.48550/arXiv.1010.2847, [1010.2847](https://arxiv.org/abs/1010.2847)
 - [32] Kanamori T, Ohara A (2010) A Bregman Extension of quasi-Newton updates II: Convergence and Robustness Properties. DOI 10.48550/arXiv.1010.2846, [1010.2846](https://arxiv.org/abs/1010.2846)
 - [33] Kanzow C, Steck D (2023) Regularization of limited memory quasi-Newton methods for large-scale nonconvex minimization. *Mathematical Programming Computation* 15(3):417–444, DOI 10.1007/s12532-023-00238-4
 - [34] Lee M, Sun Y (2025) Advancing Multi-Secant Quasi-Newton Methods for General Convex Functions. DOI 10.48550/arXiv.2504.07330, [2504.07330](https://arxiv.org/abs/2504.07330)
 - [35] Li DH, Fukushima M (2001) On the Global Convergence of the BFGS Method for Nonconvex Unconstrained Optimization Problems. *SIAM Journal on Optimization* 11(4):1054–1064, DOI 10.1137/S1052623499354242
 - [36] Li YJ, Li DH (2009) Truncated regularized Newton method for convex minimizations. *Computational Optimization and Applications* 43(1):119–131, DOI 10.1007/s10589-007-9128-7
 - [37] Liu DC, Nocedal J (1989) On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45(1):503–528, DOI 10.1007/BF01589116
 - [38] Lotfi S, de Ruisselet TB, Orban D, Lodi A (2020) Stochastic Damped L-BFGS with

- Controlled Norm of the Hessian Approximation. DOI 10.13140/RG.2.2.27851.41765/1, [2012.05783](#)
- [39] Mannel F (2025) A Globalization of L-BFGS and the Barzilai–Borwein Method for Non-convex Unconstrained Optimization. *Journal of Optimization Theory and Applications* 204(3):1–34, DOI 10.1007/s10957-024-02565-5
 - [40] Mannel F, Om Aggrawal H, Modersitzki J (2024) A structured L-BFGS method and its application to inverse problems. *Inverse Problems* 40(4):045022, DOI 10.1088/1361-6420/ad2c31
 - [41] Moré JJ, Thuente DJ (1994) Line search algorithms with guaranteed sufficient decrease. *ACM Trans Math Softw* 20(3):286–307, DOI 10.1145/192115.192132
 - [42] Moré JJ, Wild SM (2011) Estimating Computational Noise. *SIAM Journal on Scientific Computing* 33(3):1292–1314, DOI 10.1137/100786125
 - [43] Nesterov Y, Polyak B (2006) Cubic regularization of Newton method and its global performance. *Mathematical Programming* 108(1):177–205, DOI 10.1007/s10107-006-0706-8
 - [44] Nocedal J, Wright SJ (1999) *Numerical Optimization*. Springer, DOI 10.1007/978-0-387-40065-5
 - [45] Okazaki N (2007) libLBFGS: A library of limited-memory broyden-fletcher-goldfarb-shanno (l-BFGS)
 - [46] Powell MJ (1978) Algorithms for nonlinear constraints that use lagrangian functions. *Math Program* 14(1):224–248, DOI 10.1007/BF01588967
 - [47] Powell MJD (1986) How bad are the BFGS and DFP methods when the objective function is quadratic? *Mathematical Programming* 34(1):34–47, DOI 10.1007/BF01582161
 - [48] Ranganath A, Singhal M, Marcia R (2025) Symmetric Rank-One Quasi-Newton Methods for Deep Learning Using Cubic Regularization. DOI 10.48550/arXiv.2502.12298, [2502.12298](#)
 - [49] Rockafellar RT, Wets RJB (1998) *Variational Analysis*, Grundlehren Der Mathematischen Wissenschaften, vol 317. Springer, Berlin, Heidelberg, DOI 10.1007/978-3-642-02431-3
 - [50] Ruan H, Yang W (2024) Adaptive Regularized Quasi-Newton Method Using Inexact First-Order Information. *Journal of Computational Mathematics* 42(6):1656–1687, DOI 10.4208/jcm.2306-m2022-0279
 - [51] Shanno DF, Phua KH (1978) Matrix conditioning and nonlinear optimization. *Mathematical Programming* 14(1):149–160, DOI 10.1007/BF01588962
 - [52] Sheng Z, Yuan G, Cui Z (2018) A new adaptive trust region algorithm for optimization problems. *Acta Mathematica Scientia* 38(2):479–496, DOI 10.1016/S0252-9602(18)30762-8
 - [53] Shi HJM, Xie Y, Byrd R, Nocedal J (2022) A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization. *SIAM Journal on Optimization* 32(1):29–55, DOI 10.1137/20M1373190
 - [54] Ueda K, Yamashita N (2010) Convergence Properties of the Regularized Newton Method for the Unconstrained Nonconvex Optimization. *Applied Mathematics and Optimization* 62(1):27–46, DOI 10.1007/s00245-009-9094-9
 - [55] Ueda K, Yamashita N (2014) A regularized Newton method without line search for unconstrained optimization. *Computational Optimization and Applications* 59(1):321–351, DOI 10.1007/s10589-014-9656-x
 - [56] Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, Burovski E, Peterson P, Weckesser W, Bright J, van der Walt SJ, Brett M, Wilson J, Millman KJ, Mayorov N, Nelson ARJ, Jones E, Kern R, Larson E, Carey CJ, Polat I, Feng Y, Moore EW, VanderPlas J, Laxalde D, Perktold J, Cimrman R, Henriksen I, Quintero EA, Harris CR, Archibald AM, Ribeiro AH, Pedregosa F, van Mulbregt P, SciPy 10 Contributors (2020) *SciPy 1.0: Fundamental algorithms for scientific computing in python*. *Nature Methods* 17:261–272, DOI 10.1038/s41592-019-0686-2
 - [57] Wang S, Fadili J, Ochs P (2025) Convergence rates of regularized quasi-Newton methods without strong convexity. DOI 10.48550/arXiv.2506.00521, [2506.00521](#)

- [58] Ward R, Wu X, Bottou L (2020) AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. *J Mach Learn Res* 21(1):219:9047–219:9076
- [59] Wei Z, Li G, Qi L (2006) New quasi-Newton methods for unconstrained optimization problems. *Applied Mathematics and Computation* 175(2):1156–1188, DOI 10.1016/j.amc.2005.08.027
- [60] Xie Y, Byrd RH, Nocedal J (2020) Analysis of the BFGS method with errors. *SIAM Journal on Optimization* 30(1):182–209
- [61] Yabe H, Ogasawara H, Yoshino M (2007) Local and superlinear convergence of quasi-Newton methods based on modified secant conditions. *Journal of Computational and Applied Mathematics* 205(1):617–632, DOI 10.1016/j.cam.2006.05.018
- [62] Yuan G, Wei Z (2010) Convergence analysis of a modified BFGS method on convex minimizations. *Computational Optimization and Applications* 47(2):237–255, DOI 10.1007/s10589-008-9219-0
- [63] Yuan YX (1991) A Modified BFGS Algorithm for Unconstrained Optimization. *IMA Journal of Numerical Analysis* 11(3):325–332, DOI 10.1093/imanum/11.3.325
- [64] Zhang H, Ni Q (2018) A new regularized quasi-Newton method for unconstrained optimization. *Optimization Letters* 12(7):1639–1658, DOI 10.1007/s11590-018-1236-z
- [65] Zhang J, Xu C (2001) Properties and numerical performance of quasi-Newton methods with modified quasi-Newton equations. *Journal of Computational and Applied Mathematics* 137(2):269–278, DOI 10.1016/S0377-0427(00)00713-5
- [66] Zhang JZ, Deng NY, Chen LH (1999) New Quasi-Newton Equation and Related Methods for Unconstrained Optimization. *Journal of Optimization Theory and Applications* 102(1):147–167, DOI 10.1023/A:1021898630001

A

Appendix

A.1 Proof of Lemma 4

The proof of Lemma 4 is similar to existing results [58, Lemma 3.2], [26, Lemma 3.1].

Proof. Recall that eq. (3.7) defines μ_k as $\theta\sqrt{\varsigma + \sum_{k \in K^+} \|g_k\|^2}$ with K^+ at iteration k , and Line 7 of Algorithm 2 asserts $\theta \in [\theta_{\min}, \theta_{\max}]$. Let $k_i \in K^+$ be the i -th element of K^+ and define the empty sum as zero. For any $0 < x < y$, $x/y \leq -\ln(1 - x/y) = \ln y - \ln(y - x)$ holds. Thus, we have

$$\begin{aligned} \frac{\|g_{k_i}\|^2}{\mu_{k_i}^2} &\leq \frac{1}{\theta_{\min}^2} \frac{\|g_{k_i}\|^2}{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2} && \text{(definition)} \\ &\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2 \right) - \ln \left(\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2 \right) \right). && (x/y \leq \ln y - \ln(y - x)) \end{aligned}$$

Similarly, $x/\sqrt{y} \geq x/(\sqrt{y} + \sqrt{y - x}) = \sqrt{y} - \sqrt{y - x}$ holds. Thus, we have

$$\begin{aligned} \frac{\|g_{k_i}\|^2}{\mu_{k_i}} &\geq \frac{1}{\theta_{\max}} \frac{\|g_{k_i}\|^2}{\sqrt{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2}} && \text{(definition)} \\ &\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{\ell=0}^i \|g_{k_\ell}\|^2} - \sqrt{\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2} \right). && (x/\sqrt{y} \geq \sqrt{y} - \sqrt{y - x}) \end{aligned}$$

Summing up these inequalities over $0 \leq i < |K^+|$ yields the claim. $\square$ $\square$

A.2 Proof of Proposition 16

The proof of Proposition 16 is similar to existing results [20, Theorem 5.5] [22, Lemma 1].

Proof. We consider standard BFGS updates with a single curvature pair (s, y) . From eqs. (3.44) and (3.45), we have

$$\left\| \frac{yy^\top}{y^\top s} \right\| = \frac{y^\top y}{y^\top s} \leq \Lambda, \quad \left\| \frac{ys^\top}{y^\top s} \right\| \leq \frac{\|y\| \|s\|}{\sqrt{\frac{1}{\Lambda} \|y\|^2} \sqrt{\lambda \|s\|^2}} = \sqrt{\kappa}, \quad \left\| \frac{ss^\top}{y^\top s} \right\| = \frac{s^\top s}{y^\top s} \leq \frac{1}{\lambda}. \quad (\text{A.1})$$

Let B, H denote the positive definite Hessian approximation before the update and its inverse, and B_+, H_+ the updated matrices. The BFGS update rule is

$$B_+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{yy^\top}{y^\top s}, \quad H_+ = \left(I - \frac{sy^\top}{y^\top s}\right) H \left(I - \frac{ys^\top}{y^\top s}\right) + \frac{ss^\top}{y^\top s}.$$

Since B is symmetric positive definite matrix and $s \neq 0$, we have

$$\|B_+\| \leq \|B\| + \left\| \frac{yy^\top}{y^\top s} \right\| \leq \|B\| + \Lambda, \quad (\text{A.2})$$

$$\begin{aligned} \|H_+\| &= \left\| \left(I - \frac{ys^\top}{y^\top s}\right)^\top H \left(I - \frac{ys^\top}{y^\top s}\right) + \frac{ss^\top}{y^\top s} \right\| \\ &\leq \left\| I - \frac{ys^\top}{y^\top s} \right\|^2 \|H\| + \left\| \frac{ss^\top}{y^\top s} \right\| \\ &\leq \left(1 + \left\| \frac{ys^\top}{y^\top s} \right\|\right)^2 \|H\| + \left\| \frac{ss^\top}{y^\top s} \right\| \\ &\leq (1 + \sqrt{\kappa})^2 \|H\| + \frac{1}{\lambda}. \end{aligned} \quad (\text{eq. (A.1)}) \quad (\text{A.3})$$

Applying eqs. (A.2) and (A.3) recursively over p updates and using the initial matrix γI , we obtain

$$\begin{aligned} \left\| \text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}) \right\| &\leq \gamma + p\Lambda, \\ \left\| (\text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}))^{-1} \right\| &\leq (1 + \sqrt{\kappa})^{2p} \gamma + \frac{1}{\lambda} \sum_{i=0}^{p-1} (1 + \sqrt{\kappa})^{2i} \\ &= (1 + \sqrt{\kappa})^{2p} \gamma + \frac{1}{\lambda} \frac{(1 + \sqrt{\kappa})^{2p} - 1}{(1 + \sqrt{\kappa})^2 - 1} \\ &\leq (1 + \sqrt{\kappa})^{2p} \left(\gamma + \frac{1}{\lambda(2\sqrt{\kappa} + \kappa)} \right). \end{aligned}$$

Therefore, $\text{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1})$ satisfies the stated bounds in eq. (3.46). $\square$ $\square$

The conditions eqs. (3.44) and (3.45) are standard in the analysis of quasi-Newton methods [9, Theorem 2.1]. If f is L -smooth (Assumption 2) and convex, then by the Baillon–Haddad theorem [3, 5] its gradient is $1/L$ -cocoercive:

$$(\nabla f(x) - \nabla f(y))^\top (x - y) \geq \frac{1}{L} \|\nabla f(x) - \nabla f(y)\|^2 \quad (\text{A.4})$$

for all $x, y \in \mathbb{R}^n$. Applying this inequality with $x = x_{k+1}$ and $y = x_k$ yields eq. (3.44) with $\Lambda = L$.